**BỘ GIÁO DỤC VÀ ĐÀO TẠO**
**ĐẠI HỌC CẦN THƠ**
**TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**

**NIÊN LUẬN**
**NGÀNH CÔNG NGHỆ THÔNG TIN**

**Đề tài**

# Xây dựng quy trình CI/CD hoàn chỉnh cho ứng dụng Web

**Sinh viên: Trần Thanh Duy**
**Mã số: DC24V7X604**
**Khóa: 2024-2027**

*Cần Thơ, 09/2026*

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**
**ĐẠI HỌC CẦN THƠ**
**TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**
**KHOA CÔNG NGHỆ THÔNG TIN**




**NIÊN LUẬN**
**NGÀNH CÔNG NGHỆ THÔNG TIN**

**Đề tài**

# Xây dựng quy trình CI/CD hoàn chỉnh cho ứng dụng Web

**Người hướng dẫn**
**ThS. Nguyễn Minh Trung**

**Sinh viên thực hiện**
**Trần Thanh Duy**
**Mã số: DC24V7X604**
**Khóa: 2024-2027**

*Cần Thơ, 09/2026*

# LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến các thầy cô tại Đại học Cần Thơ, đặc biệt là các thầy cô trong trường Công Nghệ Thông Tin và Truyền Thông.Trong hơn một năm qua các thầy cô đã tận tình giảng dạy, hướng dẫn nhiệt tình để em có được những kiến thức mới đầy hữu ích cho tương lai sau này.

Sau gần 3 tháng tìm hiểu và thực hiện, đề tài " Xây dựng quy trình CI/CD hoàn chỉnh cho ứng dụng Web" đã hoàn thành, ngoài sự nỗ lực của bản thân, em còn nhận được nhiều sự động viên, khích lệ từ thầy cô và bạn bè.

Em xin cảm ơn Thầy Nguyễn Minh Trung đã hỗ trợ hướng dẫn em hoàn thành niên luận.

Do trình độ lý luận và kinh nghiệm thực tế còn hạn chế nên bài báo cáo còn nhiều thiếu sót, em rất mong nhận được ý kiến đóng góp từ Thầy, Cô để em có thể rút ra nhiều kinh nghiệm hơn.

Cuối cùng, em xin chân thành cảm ơn gia đình và bạn bè, đã luôn tạo điều kiện, quan tâm, giúp đỡ, động viên em trong suốt quá trình học tập và hoàn thành bài niên luận này.

Em xin chân thành cảm ơn

# MỤC LỤC

LỜI CẢM ƠN ..... I
MỤC LỤC ..... II
DANH MỤC HÌNH ..... IV
DANH MỤC BẢNG ..... V
DANH MỤC TỪ VIẾT TẮT ..... VI
TÓM TẮT ..... 1
GIỚI THIỆU ..... 2
1. Đặt vấn đề ..... 2
2. Mục tiêu đề tài ..... 4
3. Đối tượng và phạm vi nghiên cứu ..... 4
3.1. Đối tượng nghiên cứu ..... 4
3.2. Phạm vi nghiên cứu ..... 4
4. Công nghệ sử dụng ..... 5
4.1. Các công nghệ được sử dụng trong đề tài ..... 5
4.2. Giải thích các từ chuyên môn ..... 5
5. Nội dung nghiên cứu ..... 6
5.1. Nghiên cứu cơ sở lý thuyết: ..... 7
5.2. Phân tích và thiết kế quy trình (Pipeline Architecture): ..... 7
5.3. Triển khai và cấu hình hệ thống: ..... 7
5.4. Vận hành thử nghiệm và Đánh giá: ..... 7
6. Bố cục của niên luận: ..... 8
CHƯƠNG I: ĐẶC TẢ YÊU CẦU ..... 9
1.1. Tổng quan về giải pháp ..... 9
1.2. Các vấn đề cần giải quyết ..... 9
CHƯƠNG II: CƠ SỞ LÝ THUYẾT ..... 11
2.1 Tổng quan về CI/CD ..... 11
2.1.1 Tích hợp liên tục: ..... 11
2.1.2 Phân phối/triển khai liên tục ..... 12
2.2 Nền tảng tự động hóa GitHub Actions ..... 13
2.3. Công cụ quản lý dự án và kiểm tra mã nguồn Java ..... 14
2.4. Đánh giá bảo mật (Security Evaluation) ..... 14
2.5. Công nghệ ảo hóa (Docker) ..... 15
2.6. GitHub Container Registry (GHCR) ..... 15
2.7 Watchtower và cơ chế tự động cập nhật container ..... 15

CHƯƠNG III: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP ..... 17
3.1 Phân tích kiến trúc hệ thống ứng dụng Web mẫu ..... 17
3.2. Thiết kế luồng công việc tự động hóa (Pipeline Design) ..... 18
3.3. Cài đặt quy trình CI (Continuous Integration) ..... 19
3.3.1 Tạo access token trên Github ..... 19
3.3.2. Cấu hình sự kiện kích hoạt (Triggers) ..... 20
3.3.3. Cài đặt giai đoạn Tích hợp liên tục (CI) ..... 21
3.3.4. Cài đặt giai đoạn Đánh giá bảo mật DevSecOps ..... 22
3.3.5. Cài đặt giai đoạn Đóng gói đa kiến trúc (Multi-Arch Docker Build) ..... 23
3.4. Cài đặt quy trình CD (Continuous Delivery) ..... 25
3.5. Cài đặt quy trình CD (Continuous Deployment) ..... 26
3.5.1. Cài đặt máy ảo Ubuntu 24.04 bằng Vagrant ..... 26
3.5.2. Cài đặt docker-compose ..... 28
CHƯƠNG IV: ĐÁNH GIÁ KIỂM THỬ ..... 37
4.1. Mục tiêu kiểm thử ..... 37
4.2. Phạm vi kiểm thử ..... 37
4.3. Kế hoạch kiểm thử ..... 38
4.4. Các bước kiểm thử và kết quả ghi nhận ..... 38
4.4.1. Kiểm thử workflow và artifact ..... 38
4.4.2. Kiểm thử build và test Maven ..... 39
4.4.3. Kiểm thử phát hành image đa kiến trúc ..... 39
4.4.4. Kiểm thử cập nhật tự động bằng Watchtower ..... 40
4.4.5. Đánh giá kiểm thử ..... 42
TÀI LIỆU THAM KHẢO ..... 43

# DANH MỤC HÌNH

Hình 2.1 Quy trình CI/CD ..... 11
Hình 2.2: Kiến trúc của Github Actions ..... 14
Hình 3.1: Kiến trúc Web App Java ..... 18
Hình 3.2: Tạo personal access token (classic) ..... 20
Hình 3.3: Cấu hình sự kiện kích hoạt ..... 21
Hình 3.4: Job CI ..... 22
Hình 3.5: Job Security_Scan ..... 23
Hình 3.6: Job BUILD_AMD64 ..... 24
Hình 3.7: Job BUILD_ARM64 ..... 25
Hình 3.8: Job PUBLISH_TO_GHCR ..... 26
Hình 3.9: Cấu hình Vagrant ..... 27
Hình 3.10: Sơ đồ kiến trúc hệ thống Web trên nền Docker ..... 29
Hình 3.11: Cơ sở dữ liệu vprodb ..... 30
Hình 3.12: Bộ nhớ đệm vprocache01 ..... 30
Hình 3.13: Cấu hình dịch vụ RabbitMQ ..... 31
Hình 3.14: Dịch vụ Watchtower ..... 32
Hình 3.15: Dịch vụ Ứng dụng Backend ..... 32
Hình 3.16: Dịch vụ Web Proxy vproweb ..... 33
Hình 3.17: Kết quả thực thi workflow triển khai ứng dụng ..... 34
Hình 3.18: Kết quả Workflow trong VSCode ..... 35
Hình 3.19: Log của Watchtower khi cập nhật container vproapp ..... 36
Hình 4.1: Git commit ..... 38
Hình 4.2: Summary của GitHub Actions và danh sách Artifacts ..... 39
Hình 4.3: Kết quả mvn test ..... 39
Hình 4.4: Package trên GHCR ..... 40
Hình 4.5: vagrant ssh ..... 41
Hình 4.6: Log Watchtower ..... 41
Hình 4.7: Phiên bản web mới được cập nhật ..... 42

# DANH MỤC BẢNG

Bảng: Danh mục từ viết tắt..... vi
Bảng: Các công nghệ được sử dụng..... 5
Bảng 3.1: Bảng cấu hình Vagrant ..... 28

# DANH MỤC TỪ VIẾT TẮT

| Kí hiệu & Chữ viết tắt | Diễn giải |
|---|---|
| API | Application Programming Interface (Giao diện lập trình ứng dụng) |
| AWS | Amazon Web Services (Nền tảng điện toán đám mây của Amazon) |
| CD | Continuous Delivery / Continuous Deployment (Phân phối liên tục / Triển khai liên tục) |
| CI | Continuous Integration (Tích hợp liên tục) |
| CVEs | Common Vulnerabilities and Exposures (Cơ sở dữ liệu lỗ hổng bảo mật toàn cầu) |
| GHCR | GitHub Container Registry (Kho lưu trữ Container của GitHub) |
| JDK | Java Development Kit (Bộ công cụ phát triển phần mềm Java) |
| JSON | JavaScript Object Notation (Định dạng lưu trữ và trao đổi dữ liệu) |
| LTS | Long-Term Support (Phiên bản hỗ trợ dài hạn) |
| MVC | Model-View-Controller (Mô hình kiến trúc phần mềm) |
| OCI | Open Container Initiative (Tiêu chuẩn mở cho định dạng container) |
| QA | Quality Assurance (Đội ngũ/Kiểm thử chất lượng) |
| QEMU | Quick EMUlator (Trình giả lập) |
| RAM | Random Access Memory (Bộ nhớ truy cập ngẫu nhiên) |
| SQL | Structured Query Language (Ngôn ngữ truy vấn dữ liệu - đề cập trong SQL Injection) |
| UX | User Experience (Trải nghiệm người dùng) |
| vCPU | Virtual Central Processing Unit (Nhân CPU ảo) |
| XSS | Cross-Site Scripting (Lỗ hổng bảo mật tập lệnh chéo trang) |
| YAML | YAML Ain't Markup Language (Ngôn ngữ cấu hình dữ liệu) |

*Bảng: Danh mục từ viết tắt*

# TÓM TẮT

Trong bối cảnh công nghiệp phần mềm hiện đại, phương pháp phát triển linh hoạt (Agile) đang được áp dụng rộng rãi nhằm tăng tốc độ phát triển và chuyển giao tính năng mới. Tuy nhiên, tần suất cập nhật liên tục này vô hình trung tạo ra nút thắt lớn tại giai đoạn triển khai, gây áp lực lên đội ngũ vận hành và làm tăng rủi ro sai sót hệ thống do các thao tác thủ công. Để giải quyết vấn đề này, đề tài "Xây dựng hệ thống CI/CD cho ứng dụng Web" được thực hiện nhằm tối ưu hóa vòng đời phát triển phần mềm thông qua việc áp dụng tự động hóa theo mô hình DevOps.

Niên luận tập trung vào việc thiết lập một luồng Tích hợp và Phân phối/triển khai liên tục (CI/CD pipeline) cho một ứng dụng Web phát triển bằng ngôn ngữ Java. Hệ thống đề xuất sử dụng nền tảng GitHub Actions làm cốt lõi để tự động hóa toàn bộ các tác vụ: biên dịch mã nguồn (Build), thực thi kịch bản kiểm thử (Test), và đánh giá chất lượng bảo mật (Security Evaluation). Sau khi vượt qua các bước kiểm định nghiêm ngặt, hệ thống sẽ tự động đóng gói ứng dụng vào các container (Docker Build) và phân phối an toàn lên kho lưu trữ GitHub Container Registry (GHCR). Kết quả của đề tài mang lại một giải pháp chuẩn hóa, giúp giảm thiểu tối đa sự can thiệp của con người, rút ngắn thời gian phát hành và nâng cao tính ổn định, an toàn cho hệ thống.

# GIỚI THIỆU

## 1. Đặt vấn đề

Trong bối cảnh ngành công nghiệp phần mềm hiện đại, phương pháp phát triển phần mềm Agile đang trở thành tiêu chuẩn cốt lõi tại các công ty công nghệ lớn nhằm đáp ứng nhu cầu thay đổi nhanh chóng của thị trường. Thông qua việc chia nhỏ quá trình phát triển thành các chu kỳ ngắn (Sprints), đội ngũ lập trình (Developer/Dev) có thể liên tục chuyển giao các tính năng phần mềm mới một cách nhanh chóng.

Tuy nhiên, tốc độ lập trình và tần suất chuyển giao gia tăng này tạo ra một nút thắt lớn ở giai đoạn triển khai (Deployment). Nó đặt áp lực nặng nề lên đội ngũ vận hành (Operations/Ops). Ngoài việc phải đáp ứng khối lượng lớn các yêu cầu cập nhật liên tục từ đội Dev, team Ops còn phải chịu trách nhiệm đảm bảo thời gian hoạt động (uptime), tính sẵn sàng và sự ổn định của hệ thống máy chủ tổng thể.

Sự mâu thuẫn giữa tốc độ phát triển và sự ổn định trong vận hành bộc lộ rõ qua những điểm yếu của quy trình làm việc thủ công truyền thống:

- **Ở giai đoạn Phát Triển (Development):** Thông thường, các nhóm phát triển làm việc độc lập trong nhiều tuần với khối lượng mã nguồn (source code) đồ sộ. Khi toàn bộ mã nguồn này được gộp chung và đưa lên máy chủ để biên dịch (build) cùng một lúc, hệ thống thường xuyên phát sinh hàng loạt lỗi phần mềm và xung đột cấu hình. Việc phát hiện lỗi quá muộn ở giai đoạn cuối khiến các kỹ sư mất rất nhiều thời gian để rà soát và khắc phục sự cố.
- **Ở giai đoạn Triển khai (Deployment):** Việc đưa ứng dụng lên môi trường thực tế (Production) là một chuỗi các công việc phức tạp, bao gồm cung cấp máy chủ, cấu hình các thư viện phụ thuộc (dependencies), thay đổi thiết lập mạng và triển khai các gói phần mềm đóng gói (artifact như định dạng jar/war, zip/tar). Khi nhóm Ops liên tục nhận các yêu cầu triển khai thủ công nhưng lại thiếu đi tài liệu hướng dẫn cấu hình rõ ràng, rủi ro về sai sót của con người là rất lớn, dẫn đến nguy cơ gián đoạn dịch vụ và giảm trải nghiệm người dùng.

Để giải quyết triệt để sự mâu thuẫn này, văn hóa DevOps ra đời nhằm phá bỏ rào cản truyền thống, giúp đội ngũ Development và Operations làm việc gắn kết chặt chẽ với nhau. Trọng tâm của DevOps là áp dụng các công cụ tự động hóa (Automation Tools) nhằm giảm thiểu thao tác thủ công và loại bỏ sai sót từ con người. Cốt lõi của mô hình này là quy trình Tích hợp liên tục và Phân phối/triển khai liên tục (CI/CD):

- Tích hợp liên tục (Continuous Integration - CI): Yêu cầu mọi thay đổi (commit/push) mã nguồn của lập trình viên phải đi kèm với quá trình xây dựng và kiểm thử tự động ngay lập tức. Việc này giúp phát hiện và cô lập lỗi (sự cố nền, xung đột) ngay từ giai đoạn đầu để chúng không nhân lên diện rộng. Sau

khi vượt qua kiểm định, mã nguồn được đóng gói thành phần mềm hoàn chỉnh (artifact). CI hoạt động như một hệ thống kiểm tra tự động. Mỗi khi code được thay đổi và commit lên hệ thống, các bài kiểm tra tự động sẽ được chạy để kiểm tra xem code có hoạt động đúng và không gây lỗi hay không.

- Phân phối liên tục (Continuous Delivery): là quá trình tiếp nối CI, trong đó mọi phiên bản phần mềm đã vượt qua các bước build, kiểm thử và đánh giá bảo mật sẽ được tự động đóng gói, lưu trữ tại kho artifact hoặc container registry, đồng thời sẵn sàng để triển khai lên các môi trường như Testing, Staging hoặc Production. Với Continuous Delivery, việc triển khai lên Production có thể cần sự phê duyệt thủ công của người quản trị.
- Triển khai liên tục (Continuous Deployment): là mức độ tự động hóa cao hơn của Continuous Delivery. Sau khi mã nguồn vượt qua toàn bộ các bước kiểm định trong pipeline, phiên bản mới sẽ được tự động triển khai lên môi trường chạy thực tế mà không cần thao tác thủ công. Để bảo đảm an toàn, quy trình này cần kết hợp cơ chế kiểm tra tình trạng hoạt động của ứng dụng (health check), theo dõi nhật ký hệ thống, kiểm soát phiên bản image và khả năng quay lui về phiên bản ổn định trước đó khi xảy ra sự cố.

Xuất phát từ những bất cập thực tế và tiềm năng tối ưu hóa trên, đề tài "Xây dựng hệ thống CI/CD cho ứng dụng Web" được thực hiện. Niên luận tập trung vào việc tự động hóa toàn bộ vòng đời phát triển của một ứng dụng Web viết bằng ngôn ngữ Java.

Hệ thống đề xuất sẽ sử dụng nền tảng GitHub Actions để thiết lập một luồng CI/CD (pipeline) tự động hóa. Các tác vụ cốt lõi bao gồm: tự động biên dịch mã nguồn (Build), chạy các kịch bản kiểm thử (Test), đánh giá chất lượng và bảo mật (Security Evaluation), tự động đóng gói ứng dụng vào các container Docker (Docker Build) và phân phối an toàn lên kho lưu trữ GitHub Container Registry (GHCR). Việc triển khai thành công hệ thống này không chỉ giúp tối ưu hóa thời gian phát hành phần mềm mà còn trang bị môi trường hiện đại, bám sát các tiêu chuẩn thực tế đang được áp dụng tại doanh nghiệp.

Sau khi Docker image được xuất bản lên GitHub Container Registry, máy chủ Ubuntu sẽ tự động nhận biết phiên bản image mới thông qua dịch vụ Watchtower. Watchtower định kỳ kiểm tra image mới trên GHCR, tải image về máy chủ, dừng container phiên bản cũ và khởi tạo container phiên bản mới theo cấu hình đã định nghĩa. Hoàn thiện bước Continuous Deployment cho môi trường thực nghiệm.

## 2. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng thành công hệ thống tự động hóa quy trình Tích hợp liên tục, Phân phối liên tục và Triển khai liên tục (CI/CD) cho ứng dụng Web phát triển bằng ngôn ngữ Java. Hệ thống tự động hóa từ khâu biên dịch, kiểm thử, đánh giá chất lượng và bảo mật mã nguồn, đóng gói Docker image, xuất bản image lên GitHub Container Registry đến tự động cập nhật ứng dụng trên máy chủ Ubuntu. Việc triển khai nhằm giảm thiểu sự can thiệp thủ công, rút ngắn thời gian phát hành, tăng tính nhất quán giữa các môi trường và bảo đảm khả năng vận hành ổn định của hệ thống.

## 3. Đối tượng và phạm vi nghiên cứu

### 3.1. Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu các đối tượng cốt lõi sau:

Quy trình Tích hợp liên tục (Continuous Integration - CI) và Phân phối liên tục (Continuous Delivery/Deployment - CD) trong quy trình phát triển phần mềm hiện đại.

Các công nghệ và nền tảng hỗ trợ tự động hóa DevOps, cụ thể bao gồm: nền tảng chạy luồng công việc tự động GitHub Actions, công nghệ ảo hóa cấp hệ điều hành (Containerization) bằng Docker, và kho lưu trữ container GitHub Container Registry (GHCR).

Mã nguồn và cấu trúc của một ứng dụng Web được phát triển dựa trên ngôn ngữ Java.

### 3.2. Phạm vi nghiên cứu

Để đảm bảo tính khả thi và tập trung đúng vào khía cạnh vận hành (Ops), đề tài được giới hạn trong các phạm vi sau:

Về mặt ứng dụng: Đề tài không đi sâu vào việc phát triển các nghiệp vụ (business logic) phức tạp cho ứng dụng Web. Thay vào đó, đề tài sẽ sử dụng một ứng dụng Web Java (sample app) làm mẫu để tập trung trong việc thiết kế, kiểm thử và tối ưu hóa hệ thống CI/CD.

Về mặt quy trình tự động hóa: Luồng CI/CD được thiết lập trên nền tảng GitHub Actions với các tác vụ: tự động biên dịch mã nguồn (Build), chạy kiểm thử (Test), kiểm tra coding convention, đánh giá chất lượng và bảo mật (Security Evaluation), đóng gói ứng dụng thành Docker image (Docker Build), và xuất bản image lên GitHub Container Registry (GHCR). Sau khi image được xuất bản thành công, dịch vụ Watchtower trên máy ảo Ubuntu sẽ tự động kiểm tra, tải phiên bản image mới và khởi động lại container ứng dụng. Qua đó, hệ thống thực hiện Continuous Deployment trong phạm vi môi trường thực nghiệm.

Về môi trường thực nghiệm: Hệ thống được triển khai và cấu hình trên nền tảng hệ sinh thái của GitHub (bao gồm Source code repository, GitHub Actions runner, và tính năng GHCR) kết hợp với Docker Engine trên môi trường giả lập. Ngoài ra hệ thống còn có phần cấu hình Vagrant để tự động triển khai máy ảo Ubuntu để chạy hệ thống web cho phần Continuous Deployment.

Đề tài chưa đi sâu vào các bài toán quản lý cụm máy chủ lớn (như Kubernetes) hay thiết lập hạ tầng mạng phức tạp trên các nền tảng Cloud (AWS, Azure và Google Cloud).

## 4. Công nghệ sử dụng

### 4.1. Các công nghệ được sử dụng trong đề tài

| Nhóm công nghệ | Công nghệ |
|---|---|
| Quản lý mã nguồn | Git, GitHub |
| Công cụ CI/CD | Github Action |
| Build và kiểm thử | Maven |
| Kiểm tra coding convention | Checkstyle |
| Phân tích chất lượng mã nguồn | Aqua Security |
| Container | Docker (bao gồm Watchtower) |
| Container registry | Github Container Registry (GHCR) |
| Môi trường triển khai | Ubuntu 24.04 trên nền Vagrant và VMware |
| Công cụ hỗ trợ | Docker CLI |

*Bảng: Các công nghệ được sử dụng*

### 4.2. Giải thích các từ chuyên môn

**Continuous Delivery:** là quy trình tự động đưa mọi thay đổi mã nguồn đã được build, kiểm thử và kiểm tra bảo mật thành một phiên bản có thể triển khai được. Phiên bản này thường được lưu trữ trên kho artifact hoặc container registry và sẵn sàng triển khai lên môi trường Testing, Staging hoặc Production. Việc phát hành lên Production có thể yêu cầu phê duyệt thủ công.

**Continuous Deployment:** là quy trình tự động triển khai phiên bản phần mềm đã vượt qua toàn bộ kiểm định lên môi trường chạy thực tế mà không cần can thiệp thủ công. Trong đề tài này, Continuous Deployment được thực hiện bằng Watchtower:

dịch vụ này theo dõi Docker image mới trên GHCR, tự động tải image mới, thay thế container đang chạy và khởi động phiên bản ứng dụng mới trên máy ảo Ubuntu.

**Watchtower:** là công cụ hỗ trợ tự động cập nhật các Docker container. Watchtower định kỳ kiểm tra Docker registry để phát hiện image mới; khi có phiên bản mới, công cụ sẽ tải image, dừng và xóa container cũ, sau đó khởi tạo lại container mới dựa trên cấu hình hiện có. Công cụ này phù hợp với môi trường thực nghiệm có quy mô nhỏ hoặc một máy chủ đơn lẻ.

**Vagrant:** là công cụ mã nguồn mở hỗ trợ tự động hóa việc tạo, cấu hình và quản lý môi trường máy ảo. Vagrant sử dụng tệp cấu hình có tên Vagrantfile để mô tả cấu hình máy ảo như hệ điều hành, dung lượng RAM, số lượng CPU, thiết lập mạng và các bước cài đặt phần mềm. Trong đề tài, Vagrant được sử dụng để tự động khởi tạo máy ảo Ubuntu làm môi trường triển khai ứng dụng Web, cài đặt Docker Engine, Docker Compose và Watchtower. Nhờ đó, môi trường triển khai có thể được tái tạo nhất quán, nhanh chóng và giảm các sai sót khi cấu hình thủ công.

**Github Container Registry và Docker Hub:** hai nền tảng lưu trữ, quản lý và phân phối container image (thường là Docker image) phổ biến nhất hiện nay. Về cơ bản, chúng đóng vai trò như các "kho" chứa mã ứng dụng đã được đóng gói cùng toàn bộ môi trường chạy, giúp việc triển khai phần mềm diễn ra đồng nhất và nhanh chóng

**Docker Hub:** Đây là kho lưu trữ container chính thức và lâu đời nhất của Docker Inc.. Nó được cộng đồng phần mềm sử dụng rộng rãi làm điểm đến mặc định để tìm kiếm các ứng dụng và cơ sở hạ tầng dựng sẵn (như Nginx, MySQL, Redis)

**GitHub Container Registry (GHCR):** Đây là dịch vụ lưu trữ container tích hợp sẵn trong hệ sinh thái GitHub, cho phép nhà phát triển lưu trữ image ngay tại nơi họ quản lý mã nguồn. Ưu điểm nổi bật của GHCR là kết nối cực kỳ liền mạch với GitHub Actions để tự động hóa quy trình CI/CD, phân quyền quản lý chi tiết (granular permissions)

**GitHub Actions:** nền tảng tự động hóa tích hợp sẵn trên **GitHub**. Nó cho phép tạo các quy trình làm việc (workflow) để tự động hóa mọi bước trong quy trình phát triển phần mềm, bao gồm build, test và triển khai ứng dụng ngay khi có sự kiện (như push code hoặc tạo pull request) xảy ra

## 5. Nội dung nghiên cứu

Để hoàn thành các mục tiêu đề ra, đề tài được tiến hành theo trình tự các bước sau, tập trung sâu vào việc thiết lập, cấu hình và tối ưu hóa hệ thống vận hành:

**5.1. Nghiên cứu cơ sở lý thuyết:**

Tìm hiểu tổng quan về phương pháp Agile và văn hóa DevOps trong phát triển phần mềm.

Nghiên cứu nguyên lý hoạt động, ưu nhược điểm của quy trình Tích hợp và Phân phối/triển khai liên tục (CI/CD).

Khảo sát kiến trúc kỹ thuật của công nghệ Docker và nền tảng GitHub Actions.

**5.2. Phân tích và thiết kế quy trình (Pipeline Architecture):**

Thiết kế sơ đồ luồng công việc tự động (workflow) cho ứng dụng Java.

Xác định thứ tự các giai đoạn cốt lõi trong pipeline gồm: Checkout mã nguồn, Thiết lập môi trường (Setup JDK), Biên dịch (Build), Kiểm thử (Test), Kiểm tra coding convention, Quét bảo mật (Security Evaluation), Đóng gói Docker image (Docker Build), Xuất bản image lên GHCR (Publish to GHCR), và tự động triển khai phiên bản image mới lên máy ảo Ubuntu (Continuous Deployment) thông qua Watchtower.

**5.3. Triển khai và cấu hình hệ thống:**

Khởi tạo môi trường kho lưu trữ (repository) trên hệ sinh thái GitHub và đưa mã nguồn ứng dụng mẫu lên hệ thống.

Xây dựng kịch bản Dockerfile đa bước (multi-stage) để tối ưu hóa dung lượng ứng dụng Java.

Viết mã kịch bản cấu hình YAML (.github/workflows/) để tự động hóa các tác vụ.

Tích hợp công cụ Security Evaluation để tự động phân tích mã nguồn và thiết lập cơ chế chặn luồng (fail pipeline) nếu phát hiện lỗ hổng nghiêm trọng.

Thiết lập quản lý biến môi trường bí mật (GitHub Secrets) để phân quyền an toàn khi hệ thống tự động đẩy Docker Image lên kho GHCR.

Tự động phát hiện thay đổi ở mã nguồn và tự động cập nhật bằng dịch vụ Watchtower của Docker

Cấu hình Docker Compose hoặc Docker CLI trên máy ảo Ubuntu để khởi tạo container ứng dụng và dịch vụ Watchtower.

Cấu hình Watchtower xác thực với GHCR, định kỳ kiểm tra Docker image mới và tự động cập nhật container ứng dụng khi phát hiện phiên bản mới được xuất bản.

Thiết lập health check cho container ứng dụng nhằm kiểm tra trạng thái hoạt động sau khi triển khai; đồng thời lưu trữ tag phiên bản Docker image để hỗ trợ xác định và quay lại phiên bản ổn định khi cần thiết.

**5.4. Vận hành thử nghiệm và Đánh giá:**

Giả lập các thao tác của lập trình viên (commit, push code, tạo pull request) để kích hoạt tự động luồng CI/CD.

Giám sát và kiểm tra tính đúng đắn của các log kiểm thử, báo cáo quét lỗi bảo mật.

Xác minh tính khả dụng của Docker Image đầu ra trên GitHub Container Registry.

Tự động hoá việc cài đặt máy ảo Ubuntu để khởi tạo webserver và tự động phát hiện ra thay đổi ở mã nguồn và tự động cập nhật mã nguồn

Tổng hợp kết quả, đánh giá ưu/nhược điểm của hệ thống đề xuất so với phương pháp thủ công và đưa ra hướng phát triển mở rộng trong tương lai.

**6. Bố cục của niên luận:**

Niên luận được chia làm 4 chương

Chương I: Đặc tả yêu cầu

Chương II: Cơ sở lý thuyết

Chương III: Thiết kế và cài đặt hệ thống

Chương IV: Đánh giá kiểm thử

# CHƯƠNG I: ĐẶC TẢ YÊU CẦU

## 1.1. Tổng quan về giải pháp

Để đáp ứng được tốc độ phát triển phần mềm hiện đại và sự ổn định trong vận hành, giải pháp đề xuất tập trung vào việc tự động hóa toàn bộ vòng đời phát triển của một ứng dụng Web viết bằng ngôn ngữ Java. Cốt lõi của giải pháp là áp dụng văn hóa DevOps cùng các công cụ tự động hóa nhằm giảm thiểu thao tác thủ công.

Cụ thể, hệ thống đề xuất sẽ sử dụng nền tảng GitHub Actions để thiết lập một luồng Tích hợp liên tục và Phân phối liên tục (CI/CD pipeline) tự động hóa. Quy trình giải pháp được thiết kế với các giai đoạn cốt lõi như sau:

- Tự động hóa hoàn toàn các bước từ biên dịch mã nguồn (Build), chạy các kịch bản kiểm thử (Test) cho đến đánh giá chất lượng và bảo mật (Security Evaluation).
- Sau khi mã nguồn vượt qua các khâu kiểm định, luồng công việc sẽ tự động đóng gói ứng dụng vào các container Docker (Docker Build).
- Cuối cùng, các phiên bản đóng gói (Docker image) sẽ được phân phối và lưu trữ an toàn lên kho lưu trữ GitHub Container Registry (GHCR).
- Sau khi Docker image được xuất bản lên GHCR, dịch vụ Watchtower chạy trên máy ảo Ubuntu sẽ tự động phát hiện image mới, tải image về và thay thế container ứng dụng đang hoạt động. Bước này hiện thực hóa Continuous Deployment, giúp phiên bản mới được đưa vào môi trường chạy mà không yêu cầu quản trị viên thực hiện thủ công từng lần triển khai.

Thông qua giải pháp này, đề tài xây dựng một hệ thống chuẩn hóa, giúp loại bỏ các rào cản truyền thống giữa nhóm phát triển (Dev) và nhóm vận hành (Ops), đồng thời mang lại một môi trường triển khai đồng nhất, độc lập và bám sát các tiêu chuẩn công nghiệp.

## 1.2. Các vấn đề cần giải quyết

Hệ thống CI/CD được thiết kế nhằm giải quyết trực diện những điểm yếu của quy trình làm việc thủ công truyền thống. Dưới đây là các vấn đề trọng tâm mà hệ thống cần khắc phục:

- **Sự chậm trễ và xung đột trong tích hợp mã nguồn**: Thông thường, các nhóm phát triển làm việc độc lập trong nhiều tuần với khối lượng mã nguồn đồ sộ. Việc gộp chung mã nguồn và đưa lên máy chủ để biên dịch cùng lúc thường xuyên làm phát sinh hàng loạt lỗi phần mềm và xung đột cấu hình. Vấn đề này khiến kỹ sư mất rất nhiều thời gian để rà soát và khắc phục sự cố do phát hiện lỗi quá muộn

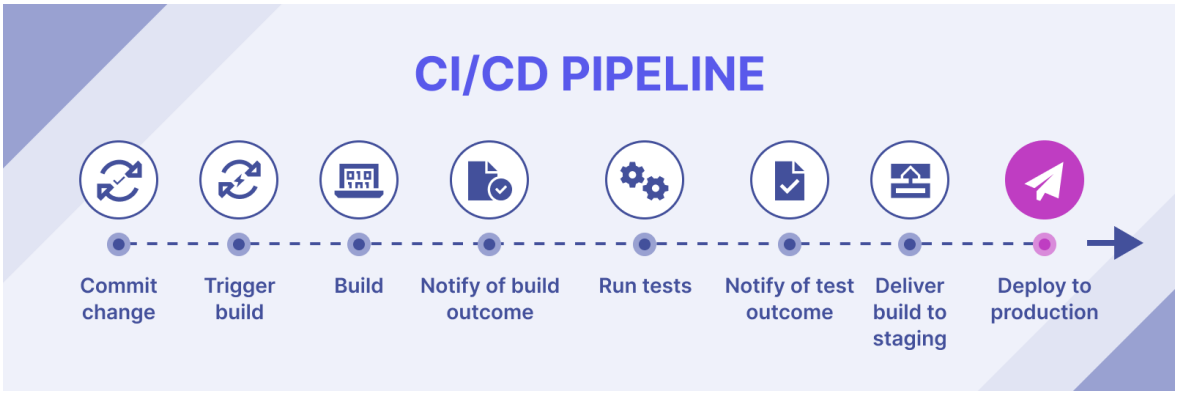
- **Rủi ro từ các thao tác triển khai thủ công**: Quá trình đưa ứng dụng lên môi trường thực tế (Production) là một chuỗi các công việc phức tạp như cung cấp máy chủ, cấu hình thư viện phụ thuộc, và triển khai các gói artifact (jar/war, zip/tar). Khi các thao tác này được thực hiện thủ công mà thiếu tài liệu cấu hình rõ ràng, rủi ro sai sót của con người là rất lớn, dễ dẫn đến gián đoạn dịch vụ và giảm trải nghiệm người dùng.
- **Áp lực quá tải lên đội ngũ vận hành (Ops)**: Tốc độ phát triển nhanh chóng (thông qua các Sprints) tạo ra nút thắt lớn ở giai đoạn triển khai. Điều này đặt áp lực nặng nề lên đội ngũ Operations, khi họ vừa phải đáp ứng khối lượng lớn yêu cầu cập nhật liên tục từ nhóm Dev, vừa phải chịu trách nhiệm đảm bảo thời gian hoạt động (uptime) và sự ổn định của hệ thống máy chủ tổng thể.

# CHƯƠNG II: CƠ SỞ LÝ THUYẾT

## 2.1 Tổng quan về CI/CD

CI/CD là một trong những tiến bộ quan trọng trong phát triển phần mềm hiện đại, giúp thay đổi cách các nhóm DevOps quản lý quá trình phát triển và triển khai sản phẩm. Ban đầu, các quy trình phát triển phần mềm thường bị chia tách với quy trình kiểm thử và triển khai, dẫn đến phát hiện lỗi muộn và khó khăn trong việc duy trì tính ổn định khi tích hợp nhiều thay đổi cùng lúc.

CI/CD (Continuous Integration/Continuous Delivery - Deployment) là một tập hợp các phương pháp và công cụ giúp tự động hóa quy trình phát triển, kiểm thử, và triển khai phần mềm. Mục tiêu chính của CI/CD là đẩy nhanh tốc độ phát hành phần mềm, giảm lỗi và nâng cao tính ổn định của các phiên bản phát hành (release).




*Hình 2.1 Quy trình CI/CD*

### 2.1.1 Tích hợp liên tục:

CI (Continuous Integration) là quá trình liên tục kiểm tra và tích hợp code mới vào code chính của một dự án. Mục tiêu của CI là đảm bảo rằng bất kỳ thay đổi nào trong code (chẳng hạn như sửa lỗi hoặc thêm tính năng) đều được kiểm tra ngay lập tức để phát hiện lỗi trước khi nó gây ảnh hưởng xấu đến hệ thống.

CI hoạt động như một hệ thống kiểm tra tự động. Mỗi khi bạn thay đổi code và commit lên hệ thống, các bài kiểm tra tự động sẽ được chạy để kiểm tra xem code có hoạt động đúng và không gây lỗi hay không.

*2.1.1.1 Những bước cơ bản trong CI*

1. **Functional tests**: Kiểm tra xem phần mềm có hoạt động đúng theo mong đợi hay không. Đây thường là các bài kiểm tra tự động mà đội ngũ QA (Quality Assurance) phát triển để đảm bảo tính năng hoạt động đúng. VD: Người dùng có thể thêm sản phẩm vào giỏ hàng và thực hiện thanh toán thành công hay không.
2. **Security scans**: Kiểm tra code có lỗ hổng bảo mật nào không, chẳng hạn như khả năng bị SQL Injection hay XSS (Cross-Site Scripting).

3. **Code quality scans**: Đảm bảo code tuân thủ các tiêu chuẩn, ví dụ như độ dài hàm, cách sử dụng khoảng trắng, và các quy tắc coding style. VD: Tiêu chuẩn như đặt tên biến.
4. **Performance tests**: Kiểm tra xem code có đáp ứng được yêu cầu về hiệu suất không, chẳng hạn như thời gian xử lý yêu cầu hoặc khả năng chịu tải.
5. **License scanning**: Kiểm tra xem tất cả các thư viện hoặc công cụ bạn sử dụng có giấy phép phù hợp hay không, để tránh các vấn đề pháp lý.
6. **Fuzz testing**: Gửi các dữ liệu bất thường, như chuỗi ký tự quá dài hoặc số không hợp lệ, vào ứng dụng để kiểm tra xem nó có bị crash hay gặp lỗi bất ngờ không.

*2.1.1.2 Lợi ích của việc thiết lập CI*

CI giúp phát hiện sớm các lỗi tiềm ẩn, tiết kiệm rất nhiều thời gian và công sức, rõ ràng việc sửa một lỗi ngay khi vừa phát hiện sẽ dễ dàng hơn nhiều so với khi lỗi đó đã tích hợp vào nhiều phần khác của hệ thống.

Ví dụ: Giả sử đội ngũ lập trình đang phát triển một tính năng mới cho ứng dụng. Thay vì đợi đến cuối quá trình phát triển để kiểm tra, CI cho phép bạn thực hiện các bài kiểm tra ngay khi lập trình viên viết code. Nhờ đó, nếu có lỗi xuất hiện, họ có thể phát hiện ngay lập tức và sửa chữa khi vấn đề còn nhỏ, thay vì đợi đến khi tính năng đã hoàn thành và tích hợp với các phần khác.

CI giúp toàn bộ đội nhóm đều có thể thấy tình trạng hiện tại của phần mềm, tất cả các thành viên từ lập trình viên, quản lý, đến QA (kiểm thử chất lượng), UX (trải nghiệm người dùng), và thậm chí cả bộ phận bảo mật có thể theo dõi tiến trình và trạng thái của dự án. Từ đó, mọi người có thể điều chỉnh công việc của mình để phù hợp với tình trạng thực tế.

Ví dụ: Khi các bài kiểm tra bảo mật được thực hiện thường xuyên, cả nhóm sẽ nhận biết được dự án có gặp phải các vấn đề bảo mật mới không. Nếu CI phát hiện rằng một đoạn code mới làm lộ lỗ hổng bảo mật, thì không chỉ các lập trình viên mà cả các quản lý dự án cũng biết ngay lập tức. Nhóm sẽ có thể phân bổ nguồn lực để sửa lỗi hoặc điều chỉnh kế hoạch phát triển.

### 2.1.2 Phân phối/triển khai liên tục

CD là quá trình đưa code từ nơi bạn viết lên một nơi mà người dùng có thể sử dụng được, như website hoặc ứng dụng di động. Có 2 loại CD chính:

- **Continuous Delivery (Phân phối liên tục):** Đảm bảo rằng code luôn sẵn sàng để được triển khai bất cứ lúc nào. Sau khi code đã vượt qua các kiểm tra CI, nó được đặt ở trạng thái sẵn sàng để triển khai. Tuy nhiên, việc triển khai có thể được thực hiện thủ công.

- **Continuous Deployment (Triển khai liên tục):** Tự động triển khai code ngay khi nó vượt qua tất cả các kiểm tra. Không cần sự can thiệp thủ công, hệ thống sẽ tự động đưa code vào môi trường production.

*2.1.2.1 Những bước cơ bản trong CD*

Build: Trước khi triển khai code, thực hiện quá trình build cho các ngôn ngữ biên dịch (compiled language) như C, C++, Java, ...

Deploy: Triển khai code: đẩy Docker image lên repository, dùng dòng lệnh để đưa code lên AWS, ... Triển khai có thể được tự động hoặc thủ công, tùy thuộc vào cách bạn thiết lập.

Ví dụ: Khi hoàn thành thêm tính năng mới như giỏ hàng, CD sẽ tự động kiểm tra và đóng gói tính năng này rồi triển khai lên website thật để khách hàng có thể sử dụng ngay lập tức.

*2.1.2.2 Lợi ích của việc thiết lập CD*

Mục tiêu của CD là giúp việc triển khai và phát hành (release) phần mềm trở nên dễ dàng và có thể diễn ra thường xuyên mà không gặp nhiều rủi ro. Khi CD được thiết lập, mỗi lần bạn commit code, code có thể tự động được triển khai vào các môi trường như: review environment, Staging environment hay production environment.

Các môi trường khác ngoài production (như review và staging) hoạt động giống như những phép thử, nhằm đảm bảo code chạy ổn định trước khi release chính thức cho người dùng.

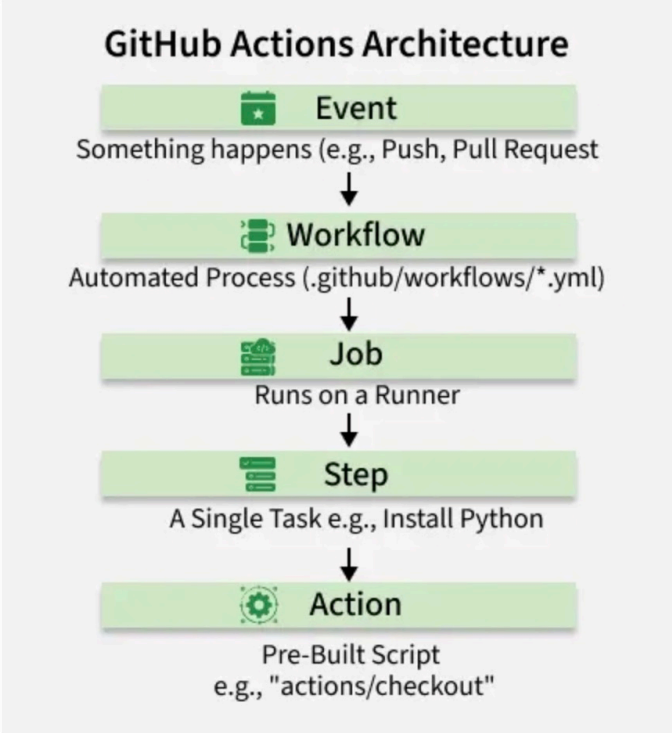
## 2.2 Nền tảng tự động hóa GitHub Actions

GitHub Actions là nền tảng tự động hóa được tích hợp sẵn trực tiếp trên hệ sinh thái GitHub. Nền tảng này cho phép tạo các quy trình làm việc (workflow) nhằm tự động hóa mọi bước trong vòng đời phát triển phần mềm, từ build, test cho đến deploy ứng dụng ngay khi có sự kiện xảy ra (như push code hoặc tạo pull request).

Trong kiến trúc của GitHub Actions, một luồng công việc tự động được định nghĩa thông qua tệp tin cấu hình YAML và bao gồm các thành phần:

- **Events (Sự kiện):** Các trigger kích hoạt quy trình, ví dụ như push hoặc pull_request trên nhánh main.
- **Jobs (Công việc):** Một luồng có thể chạy một hoặc nhiều job (như Build, Testing, Security_Scan, Publish) tuần tự hoặc song song trên các máy chủ ảo (runner) do GitHub cung cấp (ví dụ: ubuntu-latest).
- **Steps (Bước):** Từng nhiệm vụ cụ thể bên trong một job, có thể là thực thi một lệnh shell hoặc gọi các actions có sẵn (như actions/checkout để kéo mã nguồn).

- **Artifacts (Tài nguyên đầu ra):** Chức năng lưu trữ tạm thời các gói phần mềm hoặc tệp báo cáo (như file .war hoặc báo cáo bảo mật json) để chia sẻ giữa các job với nhau.




*Hình 2.2: Kiến trúc của Github Actions*

### 2.3. Công cụ quản lý dự án và kiểm tra mã nguồn Java

Ứng dụng Web trong đề tài được phát triển dựa trên ngôn ngữ Java, do đó yêu cầu các công cụ đặc thù để quản lý vòng đời và chất lượng:

- Apache Maven: Đây là công cụ quản lý dự án (quản lý thư viện phụ thuộc, quản lý vòng đời biên dịch) phổ biến nhất cho Java. Trong hệ thống, Maven đóng vai trò tải các thư viện, chạy các kịch bản kiểm thử (mvn test) và đóng gói mã nguồn thành phần mềm hoàn chỉnh định dạng WAR (mvn install).
- Checkstyle: Để đảm bảo tính đồng nhất trong phong cách lập trình và phát hiện sớm các đoạn code không tuân thủ tiêu chuẩn, công cụ Checkstyle được tích hợp vào bước kiểm thử (mvn checkstyle:checkstyle) làm rào chắn (coding convention validator).

### 2.4. Đánh giá bảo mật (Security Evaluation)

Đề tài sử dụng **Trivy (Aqua Security)**, một công cụ quét lỗ hổng bảo mật mã nguồn mở toàn diện. Trivy phân tích cấu trúc tệp tin (filesystem scan) của toàn bộ dự án để đối chiếu hệ điều hành và các thư viện phụ thuộc với các cơ sở dữ liệu lỗ

hổng bảo mật toàn cầu (CVEs). Kết quả quét được xuất ra dưới định dạng JSON và có cấu hình chặn đứng toàn bộ tiến trình CI/CD nếu phát hiện lỗ hổng nghiêm trọng

### 2.5. Công nghệ ảo hóa (Docker)

Docker là nền tảng phần mềm cho phép các nhà phát triển xây dựng, kiểm thử và triển khai các ứng dụng thông qua công nghệ ảo hóa ở dạng container. Docker đóng gói ứng dụng Web Java cùng toàn bộ môi trường chạy thực tế (runtime, thư viện hệ thống) thành các đơn vị chuẩn hóa gọi là Container (Container). Nhờ vậy, ứng dụng có thể chạy đồng nhất trên bất kỳ hạ tầng nào mà không gặp tình trạng xung đột môi trường. Trong dự án, quá trình này được thực thi thông qua kịch bản Dockerfile đa bước (multi-stage) và lệnh docker build.

### 2.6. GitHub Container Registry (GHCR)

GitHub Container Registry và Docker Hub là hai nền tảng lưu trữ, quản lý và phân phối container image phổ biến. Tuy nhiên, thay vì sử dụng AWS ECR hay Docker Hub, đề tài lựa chọn GHCR vì đây là dịch vụ được tích hợp sẵn trong hệ sinh thái GitHub.

Việc sử dụng GHCR mang lại những lợi ích cốt lõi cho hệ thống:

- Cho phép nhà phát triển lưu trữ Docker image ngay tại nơi họ đang quản lý mã nguồn.
- Kết nối cực kỳ liền mạch và có độ tin cậy cao với GitHub Actions.
- Đảm bảo an toàn bảo mật thông qua cơ chế phân quyền chi tiết (granular permissions) và việc xác thực tự động không cần chia sẻ mật khẩu tĩnh (secretless authentication) thông qua token mặc định của GitHub.

### 2.7 Watchtower và cơ chế tự động cập nhật container

Watchtower là công cụ hỗ trợ tự động cập nhật Docker container. Công cụ này chạy như một container trên máy chủ triển khai và định kỳ kiểm tra Docker registry để phát hiện image mới của các container được theo dõi.

Trong hệ thống của đề tài, Watchtower được cấu hình trên máy ảo Ubuntu có cài Docker Engine. Sau khi GitHub Actions xuất bản Docker image mới lên GHCR, Watchtower sẽ xác thực với GHCR, tải image mới về máy chủ, dừng container ứng dụng cũ và khởi động lại container mới từ image vừa tải.

Cơ chế này hoàn thiện giai đoạn Continuous Deployment trong phạm vi nghiên cứu. Luồng triển khai tổng quát của hệ thống được mô tả như sau:

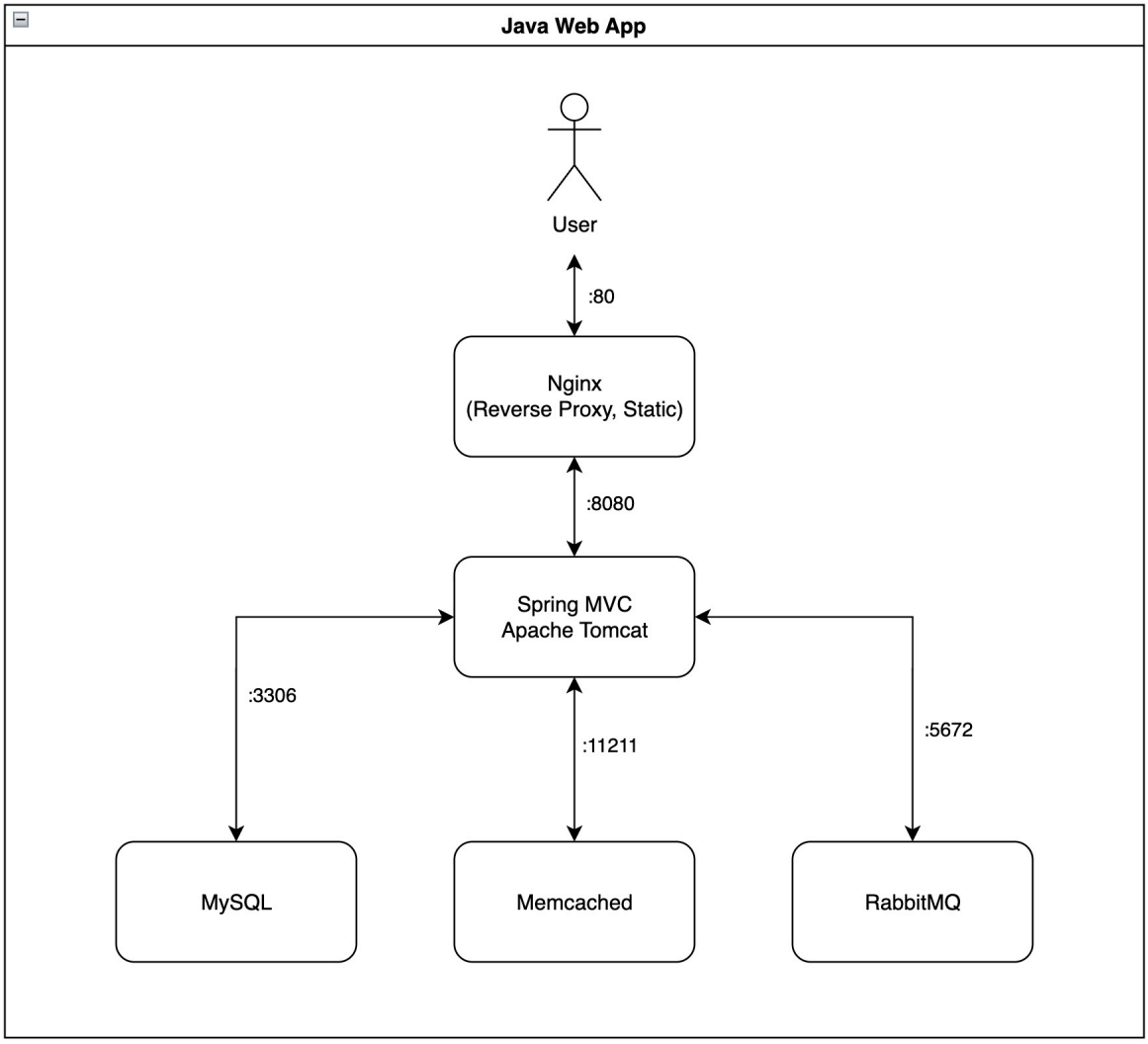
Push source code → GitHub Actions build/test/checkstyle/security scan → Docker Build → Push Docker image lên GHCR → Watchtower phát hiện image mới → Pull image → Khởi động lại container → Health check ứng dụng.

# CHƯƠNG III: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP

## 3.1 Phân tích kiến trúc hệ thống ứng dụng Web mẫu

Ứng dụng web mẫu trong đề tài (VProfile) được viết bằng ngôn ngữ lập trình Java và được thiết kế theo mô hình đa tầng (Multi-tier) bao gồm các thành phần:

- Tầng ứng dụng web(Web Tier): Sử dụng Nginx làm preverse proxy và cân bằng tải (load balancer)
- Tầng Ứng dụng (Application Tier): Được viết bằng ngôn ngữ Java và Framework Spring, chạy trên máy chủ Apache Tomcat (port 8080). Đồng thời đóng gói phần mềm đầu ra của tầng này là một tệp định dạng .war.
- Tầng Cơ sở dữ liệu và Dịch vụ hỗ trợ: Bao gồm MySQL 8.0, Memcached (caching), và RabbitMQ (Message Broker).
- Hệ thống CI/CD trong đề tài sẽ tập trung xử lý tự động hóa cho Tầng Ứng dụng web (Java Spring MVC), đảm bảo mã nguồn được kiểm thử, đóng gói thành file .war, và cuối cùng xây dựng thành Docker image để sẵn sàng tích hợp với các tầng dịch vụ khác.




*Hình 3.1: Kiến trúc Web App Java*

### 3.2. Thiết kế luồng công việc tự động hóa (Pipeline Design)

Để tối ưu hóa thời gian chạy và đảm bảo tính chặt chẽ, pipeline được chia thành 5 công việc (Jobs) chính, hoạt động kết hợp giữa tuần tự và song song:

1. Job CI (Build, Test & Package): Chạy đầu tiên để kiểm thử mã nguồn và đóng gói thành một artifact (.war).
2. Job Security_Scan: Hoạt động song song với Job CI nhằm đánh giá bảo mật mà không làm kéo dài thời gian chờ đợi.
3. Job BUILD_AMD64 & BUILD_ARM64: Chỉ được kích hoạt khi cả hai khâu kiểm thử và bảo mật đều thành công. Hai job này chạy song song trên các hạ tầng khác nhau để đóng gói Docker cho cả hai kiến trúc vi xử lý (x86_64 và ARM64).

4. Job PUBLISH_TO_GHCR: Hoạt động cuối cùng, gộp kết quả của hai tiến trình build trên thành một container đa kiến trúc (Multi-arch) và phân phối lên kho lưu trữ mã nguồn của Github (GHCR).
5. Sau khi image được xuất bản thành công, dịch vụ Watchtower trên máy ảo Ubuntu sẽ tự động kiểm tra, tải phiên bản image mới và khởi động lại container ứng dụng. Qua đó, hệ thống thực hiện Continuous Deployment trong phạm vi môi trường thực nghiệm.

### 3.3. Cài đặt quy trình CI (Continuous Integration)

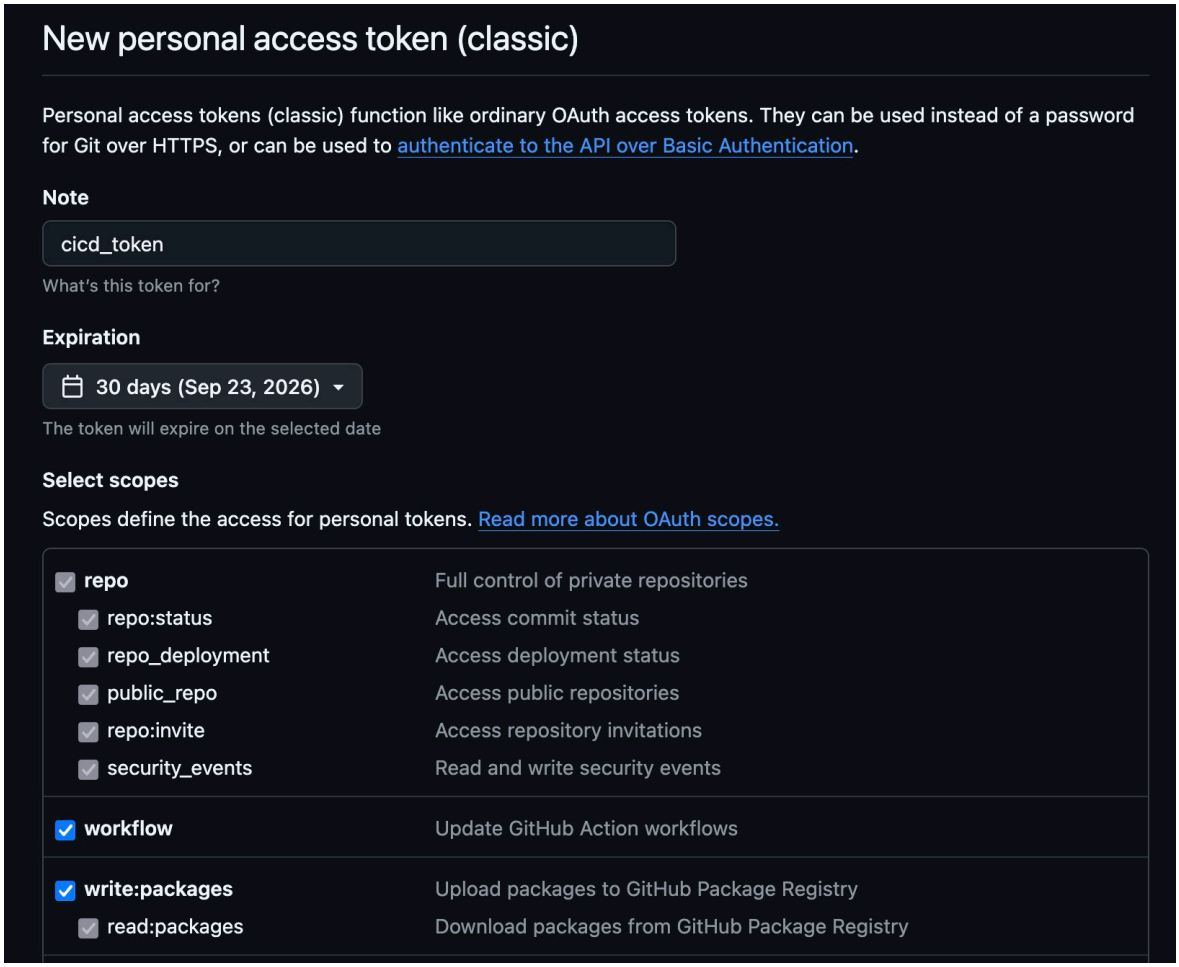
Quá trình cài đặt giải pháp được thực hiện thông qua tệp tin cấu hình YAML (.github/workflows/main.yml) đặt tại kho lưu trữ mã nguồn.

#### 3.3.1 Tạo access token trên Github

Tạo **Personal access tokens** để đăng nhập vào GHCR (GitHub Container Registry)

Các quyền cần thiết:

- Quyền quản lý repo
- Quyền cập nhật workflow
- Quyền ghi và đọc packages




*Hình 3.2: Tạo personal access token (classic)*

### 3.3.2. Cấu hình sự kiện kích hoạt (Triggers)

Hệ thống được thiết lập cơ chế tự động kích hoạt (trigger) khi có sự kiện đẩy mã nguồn (push) hoặc tạo yêu cầu gộp mã (pull_request) vào nhánh chính (main). Nền tảng được cấp quyền contents: read để sao chép mã nguồn và packages: write để đẩy các container lên GitHub Container Registry (GHCR).




*Hình 3.3: Cấu hình sự kiện kích hoạt*

### 3.3.3. Cài đặt giai đoạn Tích hợp liên tục (CI)

Giai đoạn Tích hợp liên tục chịu trách nhiệm biên dịch và kiểm thử ứng dụng Java.

- Thiết lập môi trường: Khởi tạo máy chủ ảo ubuntu-latest, cài đặt JDK 17 (Temurin). Đặc biệt, cơ chế bộ nhớ đệm (cache: 'maven') được kích hoạt để lưu lại các thư viện phụ thuộc, giúp giảm thiểu thời gian tải lại các package trong các lần chạy sau.
- Biên dịch và Kiểm thử: Lệnh mvn install được thực thi để compile, test, và package ra tệp .war.
- Kiểm tra mã nguồn: Lệnh mvn checkstyle:checkstyle được gọi để đảm bảo mã nguồn tuân thủ các quy tắc lập trình (coding convention).
- Lưu trữ Artifact đầu ra: Gói phần mềm vprofile-v2.war được trích xuất và tải lên bộ nhớ tạm của GitHub Actions (thời hạn 7 ngày) để tiết kiệm tài nguyên.

```yaml
jobs:
  # CI: Build, Test, and Package — runs once, produces a single WAR artifact
  # mvn install = compile + test + package in one Maven lifecycle
  # Checkstyle is added here so tests and style checks share the same run
  CI:
    name: Build, Test & Package
    runs-on: ubuntu-latest
    steps:
      - name: Code checkout
        uses: actions/checkout@v5

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: "temurin"
          java-version: "17"
          cache: "maven"

      - name: Build, test and package (mvn install)
        # compile → test → package all in one Maven lifecycle.
        # No need for a separate 'mvn test' step afterward.
        run: mvn install

      - name: Checkstyle
        run: mvn checkstyle:checkstyle

      - name: Upload WAR artifact
        uses: actions/upload-artifact@v4
        with:
          name: vprofile-app
          path: target/*.war
          # Retain artifact for 7 days (default is 90)
          retention-days: 7
```

*Hình 3.4: Job CI*

### 3.3.4. Cài đặt giai đoạn Đánh giá bảo mật DevSecOps

Tuân thủ nguyên tắc "Shift-Left Security", công việc Security_Scan được cấu hình chạy song song độc lập với tiến trình CI.

Hệ thống sử dụng action aquasecurity/trivy-action để phân tích tĩnh toàn bộ hệ thống tập tin (filesystem), tìm kiếm các lỗ hổng của thư viện và hệ điều hành dựa trên dữ liệu CVE. Kết quả báo cáo được xuất ra dưới định dạng JSON làm cơ sở đánh giá chất lượng.

```yaml
# Security scan runs in parallel with CI — no need to wait for the build
Security_Scan:
  name: Security Scan (Trivy)
  runs-on: ubuntu-latest
  steps:
    - name: Code checkout
      uses: actions/checkout@v5

    - name: Run Trivy filesystem scan
      uses: aquasecurity/trivy-action@v0.36.0
      with:
        scan-type: "fs"
        scan-ref: "."
        format: json
        # exit-code:
        # 0 : continue if vulnerabilities are found
        # 1 : the workflow will be failed if vulnerabilities are found
        exit-code: 0
        vuln-type: os,library
        output: trivy-results.json
        # trivy-config: trivy.yaml

    - name: Upload Trivy scan results as artifact
      uses: actions/upload-artifact@v4
      with:
        name: trivy-scan-results
        path: trivy-results.json
```

*Hình 3.5: Job Security_Scan*

### 3.3.5. Cài đặt giai đoạn Đóng gói đa kiến trúc (Multi-Arch Docker Build)

Thay vì tự biên dịch lại mã nguồn bên trong Docker (gây dư thừa tài nguyên), hệ thống áp dụng nguyên tắc "Build Once" (Chỉ biên dịch một lần).

Các Job đóng gói sẽ tải tệp .war từ bước CI về. Dockerfile chỉ cần thực hiện lệnh COPY tệp tin này vào máy chủ Tomcat.

Để hỗ trợ tính tương thích cho nhiều nền tảng, mã nguồn được build đồng thời cho hai kiến trúc:

- AMD64: Thực thi trên runner mặc định ubuntu-latest.

```yaml
BUILD_AMD64:
  name: Build AMD64
  runs-on: ubuntu-latest
  environment: ghcr
  needs: [CI, Security_Scan]
  if: github.ref == 'refs/heads/main'
  outputs:
    digest: ${{ steps.build.outputs.digest }}
  steps:
    - name: Code checkout
      uses: actions/checkout@v5

    - name: Download pre-built WAR artifact
      uses: actions/download-artifact@v4
      with:
        name: vprofile-app
        path: target

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GHCR_ACCESS_KEY }}

    - name: Build and push AMD64 image by digest
      id: build
      uses: docker/build-push-action@v6
      with:
        context: .
        # Non-multistage Dockerfile: just COPYs the pre-built WAR.
        # Maven does NOT run again here.
        file: Docker-files/app/Dockerfile
        platforms: linux/amd64
        outputs: type=image,name=ghcr.io/${{ github.repository }},push-by-digest=true,name-canonical=true,push=true
```

*Hình 3.6: Job BUILD_AMD64*

- ARM64: Thực thi trực tiếp trên hạ tầng Native ARM (ubuntu-24.04-arm). Việc không sử dụng giả lập QEMU (Quick EMUlator) giúp tốc độ build phần mềm tăng lên đáng kể.

```yaml
BUILD_ARM64:
  name: Build ARM64
  runs-on: ubuntu-24.04-arm
  environment: ghcr
  needs: [CI, Security_Scan]
  if: github.ref == 'refs/heads/main'
  outputs:
    digest: ${{ steps.build.outputs.digest }}
  steps:
    - name: Code checkout
      uses: actions/checkout@v5

    - name: Download pre-built WAR artifact
      uses: actions/download-artifact@v4
      with:
        name: vprofile-app
        path: target

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GHCR_ACCESS_KEY }}

    - name: Build and push ARM64 image by digest
      id: build
      uses: docker/build-push-action@v6
      with:
        context: .
        # Non-multistage Dockerfile: just COPYs the pre-built WAR.
        # Maven does NOT run again here.
        file: Docker-files/app/Dockerfile
        platforms: linux/arm64
        outputs: type=image,name=ghcr.io/${{ github.repository }},push-by-digest=true,name-canonical=true,push=true
```

*Hình 3.7: Job BUILD_ARM64*

- Thay vì gán nhãn latest ngay lúc này, cả hai Job đều đẩy (push) image lên GHCR dưới dạng chuỗi mã băm (Digest) để tránh xung đột dữ liệu.

### 3.4. Cài đặt quy trình CD (Continuous Delivery)

Sau khi các container đơn lẻ hoàn tất, công việc PUBLISH_TO_GHCR sử dụng công cụ docker buildx imagetools để ghép nối hai chuỗi mã băm (Digest của AMD64 và ARM64) thành một OCI Manifest duy nhất.

Kết quả đầu ra là một Docker Image hoàn chỉnh trên kho lưu trữ GHCR. Người dùng máy chủ truyền thống hay các dòng vi xử lý kiến trúc ARM (như Apple Silicon) khi dùng docker pull ứng dụng đều sẽ nhận được container tương thích tự động.

```yaml
PUBLISH_TO_GHCR:
  name: Publish to GHCR
  runs-on: ubuntu-latest
  environment: ghcr
  needs: [BUILD_AMD64, BUILD_ARM64]
  steps:
    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GHCR_ACCESS_KEY }}

    - name: Create and push multi-arch manifest
      env:
        IMAGE_NAME: ghcr.io/${{ github.repository }}
        AMD64_DIGEST: ${{ needs.BUILD_AMD64.outputs.digest }}
        ARM64_DIGEST: ${{ needs.BUILD_ARM64.outputs.digest }}
      run: |
        docker buildx imagetools create \
          --tag $IMAGE_NAME:${{ github.sha }} \
          --tag $IMAGE_NAME:latest \
          $IMAGE_NAME@$AMD64_DIGEST \
          $IMAGE_NAME@$ARM64_DIGEST
```

*Hình 3.8: Job PUBLISH_TO_GHCR*

## 3.5. Cài đặt quy trình CD (Continuous Deployment)

### 3.5.1. Cài đặt máy ảo Ubuntu 24.04 bằng Vagrant

**Vagrant được cấu hình như sau:**

- **Vagrant.configure("2"):** Sử dụng cấu hình theo chuẩn phiên bản API v2 của Vagrant.
- **config.vm.box = "bento/ubuntu-24.04":** Sử dụng hệ điều hành Ubuntu 24.04 LTS được đóng gói bởi Bento

- **virtualbox.memory = 4096:** Cấp phát 4096 MB (4 GB) bộ nhớ RAM cho máy ảo để đáp ứng nhu cầu build image và chạy nhiều container cùng lúc.
- **virtualbox.cpus = 4:** Phân bổ 4 nhân CPU ảo (vCPU) giúp tăng tốc độ build và xử lý tác vụ của ứng dụng.
- **Cổng 80 (Guest) → 8080 (Host):** Cho phép truy cập vào Reverse Proxy (ví dụ: Nginx) từ trình duyệt máy thật thông qua địa chỉ http://localhost:8080.
- **Cổng 8080 (Guest)→ 8081 (Host):** Cho phép truy cập trực tiếp vào dịch vụ backend/ứng dụng qua địa chỉ http://localhost:8081.
- auto_correct: true: Tự động chuyển sang một cổng khả dụng khác trên máy Host nếu cổng được chỉ định (8080 hoặc 8081) đang bị ứng dụng khác chiếm dụng.
- config.vm.provision: tự động hoá cài đặt và khởi tạo dịch vụ docker

```
Vagrant.configure("2") do |config|
  config.vm.box = "bento/ubuntu-24.04"
  config.vm.box_version = "202510.26.0"

  config.vm.provider "virtualbox" do |virtualbox|
    virtualbox.memory = 4096
    virtualbox.cpus = 4
  end

  # Reach the reverse proxy at http://localhost:8080 and the app directly at
  # http://localhost:8081 from the host machine.
  config.vm.network "forwarded_port", guest: 80, host: 8080, auto_correct: true
  config.vm.network "forwarded_port", guest: 8080, host: 8081, auto_correct: true

  config.vm.provision "shell", privileged: true, inline: <<-SHELL
    set -e
    export DEBIAN_FRONTEND=noninteractive

    apt-get update
    apt-get install -y docker.io docker-compose-v2

    systemctl enable --now docker
    usermod -aG docker vagrant

    # The project is available through Vagrant's default synced folder.
    cd /vagrant
    export $(grep -v '^#' .env | xargs)
    echo $GHCR_TOKEN | docker login ghcr.io -u $GHCR_USERNAME --password-stdin
    docker compose up --build --detach
  SHELL
end
```

*Hình 3.9: Cấu hình Vagrant*

| Thành phần | Cấu hình | Mục đích / Chức năng |
|---|---|---|
| Virtualization Provider | VMWare | Nền tảng ảo hóa máy ảo |
| Base OS | bento/ubuntu-24.04 (v202510.26.0) | Hệ điều hành Ubuntu 24.04 LTS |
| CPU / RAM | 4 vCPUs / 4096 MB RAM | Đảm bảo hiệu năng cho Docker và các dịch vụ |
| Port 80 (Guest) | 8080 (Host) | Truy cập Reverse Proxy (http://localhost:8080) |
| Port 8080 (Guest) | 8081 (Host) | Truy cập trực tiếp Backend (http://localhost:8081) |
| Shared Directory | /vagrant ↔ Thư mục dự án | Đồng bộ mã nguồn theo thời gian thực |
| Container Engine | docker.io, docker-compose-v2 | Quản lý và thực thi môi trường container |
| Registry Authentication | GitHub Container Registry (ghcr.io) | Xác thực để kéo/đẩy container image |

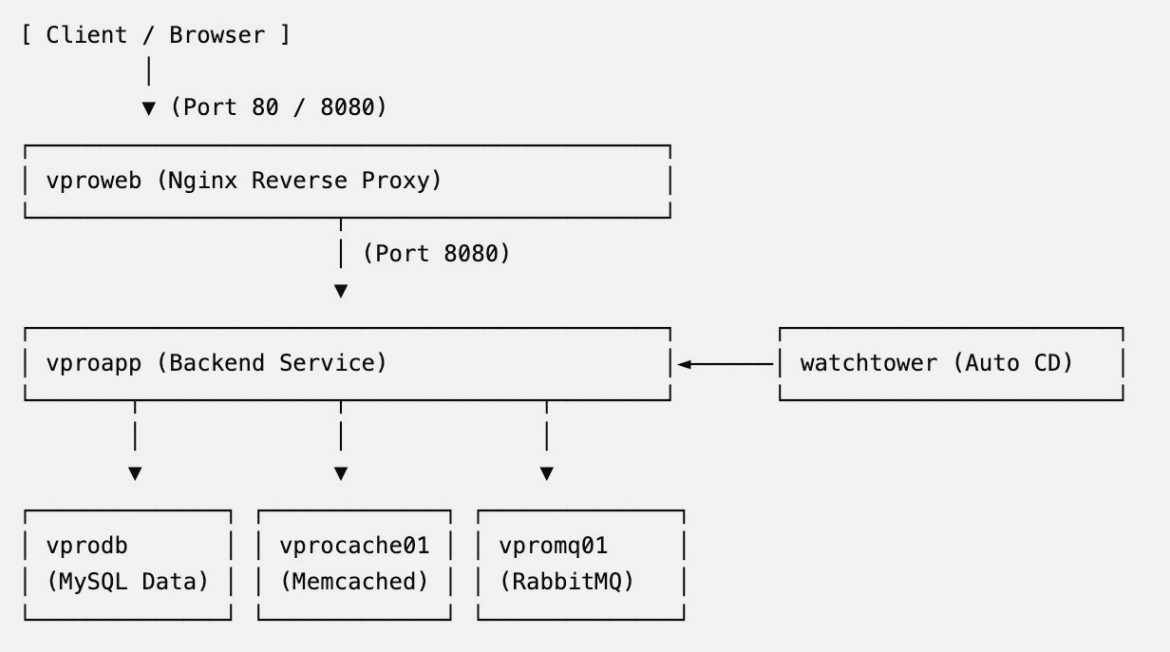
*Bảng 3.1: Bảng cấu hình Vagrant*

### 3.5.2. Cài đặt docker-compose

*3.5.2.1. Tổng quan kiến trúc hệ thống*

**Hệ thống được chia thành các lớp chức năng sau:**

- **Lớp Web (Presentation/Proxy Layer):** vproweb (Nginx).
- **Lớp Ứng dụng (Application Layer):** vproapp (Java Web/Backend App).
- **Lớp Middleware & Cache:** vpromq01 (RabbitMQ), vprocache01 (Memcached).
- **Lớp Dữ liệu (Database Layer):** vprodb (MySQL Database).
- **Lớp Tự động hóa CD (Continuous Deployment Layer):** watchtower (Tự động cập nhật phiên bản mới).




*Hình 3.10: Sơ đồ kiến trúc hệ thống Web trên nền Docker*

*3.5.2.1. Cấu hình dịch vụ Cơ sở dữ liệu vprodb*

**Dịch vụ cơ sở dữ liệu vprodb được cấu hình như sau:**

- **build.context:** Xây dựng image từ Dockerfile trong thư mục ./Docker-files/db (chứa các script khởi tạo schema database ban đầu).
- **ports (3306:3306):** Mở cổng chuẩn của MySQL để cho phép kết nối từ trong mạng container hoặc từ bên ngoài.
- **volumes (vprodbdata:/var/lib/mysql):** Gắn volume vprodbdata vào thư mục lưu trữ dữ liệu của MySQL. Đảm bảo tính toàn vẹn và bền vững của dữ liệu (Data Persistence), dữ liệu không bị mất khi container bị xóa hoặc khởi động lại.
- **environment:** Thiết lập mật khẩu lấy từ biến môi trường cho tài khoản root của cơ sở dữ liệu

```yaml
services:
  ▷Run Service
  vprodb:
    build:
      context: ./Docker-files/db
    image: vprocontainers/vprofiledb
    container_name: vprodb
    ports:
      - "3306:3306"
    volumes:
      - vprodbdata:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
```

*Hình 3.11: Cơ sở dữ liệu vprodb*

*3.5.2.2. Cấu hình dịch vụ Bộ nhớ đệm vprocache01*

**Dịch vụ Bộ nhớ đệm vprocache01 được cấu hình như sau:**

- **image: memcached:** Sử dụng image Memcached chính thức từ Docker Hub.
- **Mục đích:** Cung cấp cơ chế lưu trữ đệm trong RAM (In-memory Caching) tại cổng 11211, giúp giảm tải truy vấn cho cơ sở dữ liệu và tăng tốc độ phản hồi cho người dùng.

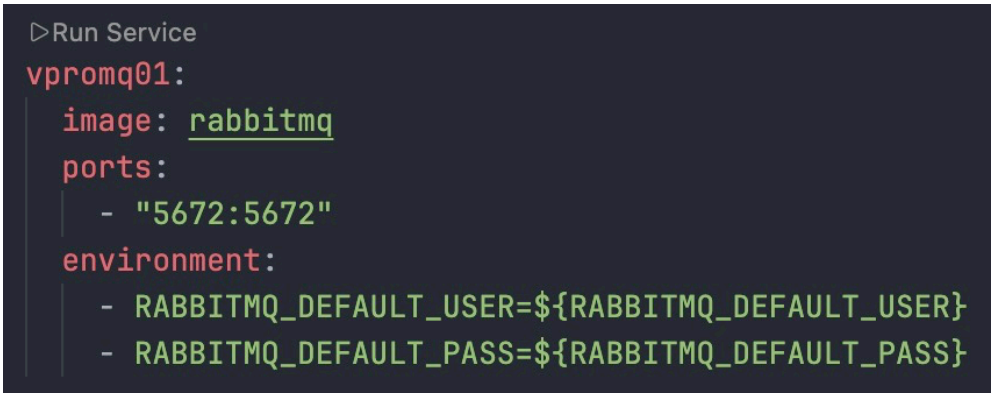
```yaml
vprocache01:
  image: memcached
  ports:
    - "11211:11211"
```

*Hình 3.12: Bộ nhớ đệm vprocache01*

*3.5.2.3. Cấu hình dịch vụ RabbitMQ*

**Dịch vụ RabbitMQ được cấu hình như sau:**

- **image rabbitmq:** Sử dụng hệ thống Message Broker RabbitMQ.
- **ports (5672:5672):** Cổng giao thức AMQP dùng để truyền/nhận các bản tin (message) giữa các thành phần.
- **Mục đích:** Hỗ trợ xử lý bất đồng bộ (Asynchronous Processing) và phân tách phụ thuộc (Decoupling) giữa các module tác vụ.




*Hình 3.13: Cấu hình dịch vụ RabbitMQ*

*3.5.2.4. Cấu hình dịch vụ Watchtower*

Dịch vụ Watchtower được cấu hình nhằm tự động phát hiện và triển khai image mới của ứng dụng Backend.

- **image:** Sử dụng image nickfedor/watchtower:1.21.0 để theo dõi phiên bản image của các container được chỉ định.
- **restart:** Thiết lập unless-stopped để dịch vụ tự khởi động lại khi Docker daemon hoặc máy chủ khởi động lại, trừ khi dịch vụ được dừng chủ động.
- **/var/run/docker.sock:/var/run/docker.sock:** Gắn Docker socket của máy Host để Watchtower có thể kiểm tra image, dừng và tạo lại container khi có bản cập nhật.
- **/root/.docker/config.json:/config.json:ro:** Gắn thông tin xác thực Docker ở chế độ chỉ đọc, cho phép Watchtower truy cập image private từ GitHub Container Registry.
- **--interval 60:** Thực hiện kiểm tra registry theo chu kỳ 60 giây để phát hiện image tag latest mới.
- **--cleanup:** Tự động dọn dẹp các image cũ sau khi container được cập nhật, giúp tiết kiệm dung lượng lưu trữ.
- **--debug:** Bật log chi tiết để hỗ trợ theo dõi tiến trình và gỡ lỗi quá trình cập nhật.
- **vproapp:** Giới hạn phạm vi theo dõi cho container vproapp, tránh cập nhật ngoài ý muốn đối với các dịch vụ hạ tầng khác.

```
▷Run Service
watchtower:
  image: nickfedor/watchtower:1.21.0
  restart: unless-stopped
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - /root/.docker/config.json:/config.json:ro
  command: --interval 60 --cleanup --debug vproapp
```

*Hình 3.14: Dịch vụ Watchtower*

*3.5.2.5. Cấu hình dịch vụ Ứng dụng Backend*

**Dịch vụ Ứng dụng Backend được cấu hình như sau:**

- **image:** Kéo image ứng dụng trực tiếp từ GitHub Container Registry (GHCR) (ghcr.io/duytechie/actionprofile:latest).
- **labels:** Đánh dấu nhãn enable: "true" để dịch vụ Watchtower nhận diện và theo dõi việc cập nhật image mới.
- **depends_on:** Chỉ định thứ tự phụ thuộc. vproapp sẽ chỉ được khởi động sau khi 3 dịch vụ nền tảng (vprodb, vprocache01, vpromq01) đã được khởi chạy.

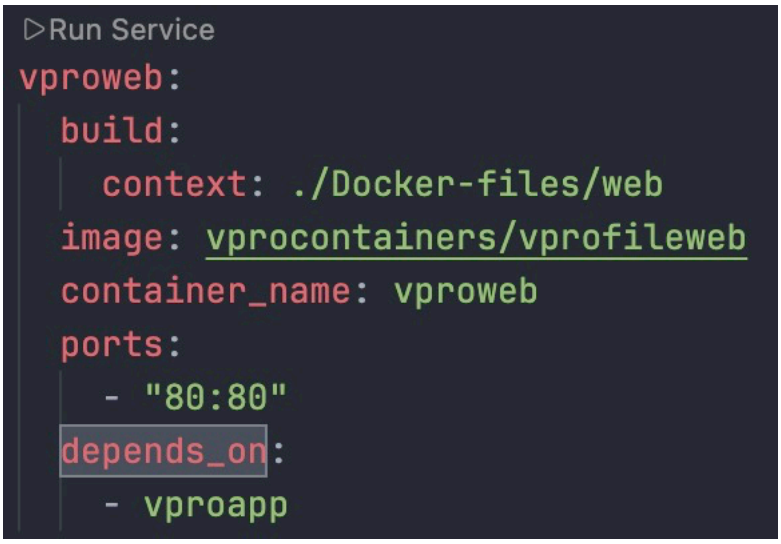
```
▷Run Service
vproapp:
  image: ghcr.io/duytechie/actionprofile:latest
  labels:
    com.centurylinklabs.watchtower.enable: "true"
  container_name: vproapp
  ports:
    - "8080:8080"
  depends_on:
    - vprodb
    - vprocache01
    - vpromq01
```

*Hình 3.15: Dịch vụ Ứng dụng Backend*

*3.5.2.6. Cấu hình dịch vụ Web Proxy vproweb*

**Dịch vụ Web Proxy vproweb được cấu hình như sau:**

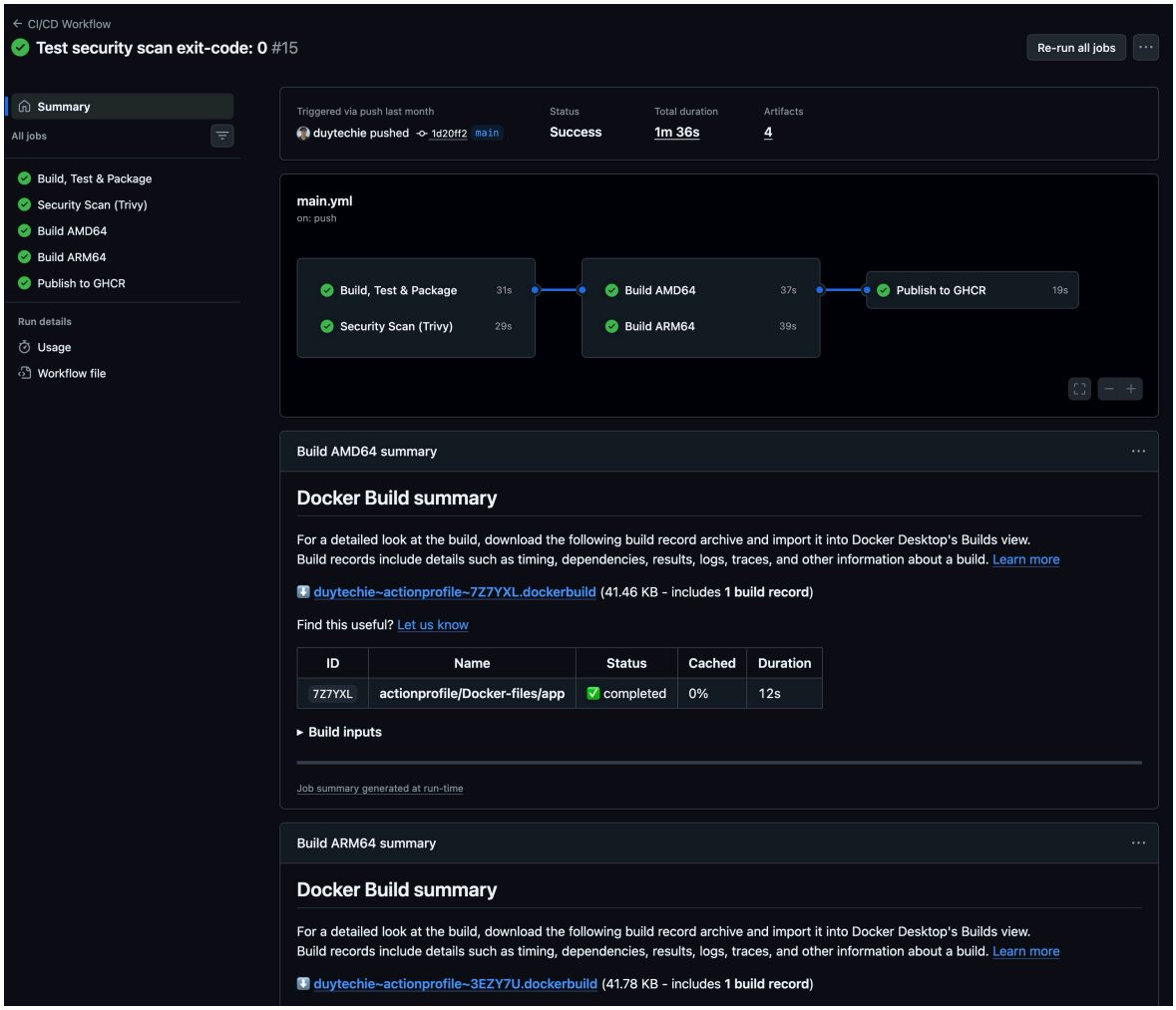
- **build.context:** Xây dựng image từ thư mục ./Docker-files/web. Dockerfile sử dụng image Nginx làm nền tảng, xoá cấu hình mặc định và sao chép file nginvproapp.conf vào thư mục cấu hình của Nginx.
- **image:** Đặt tên image sau khi build là vprocontainers/vprofileweb để thuận tiện cho việc quản lý và triển khai lại dịch vụ.
- **ports (80:80):** Công khai cổng HTTP 80 của container ra máy Host, tạo điểm truy cập web cho người dùng.
- **nginvproapp.conf:** Khai báo upstream vproapp trỏ đến vproapp:8080 và sử dụng proxy_pass để chuyển tiếp mọi yêu cầu từ Nginx đến ứng dụng Backend thông qua mạng nội bộ Docker.
- **depends_on:** Chỉ định vproweb khởi động sau vproapp, bảo đảm lớp proxy được triển khai sau dịch vụ ứng dụng Backend.



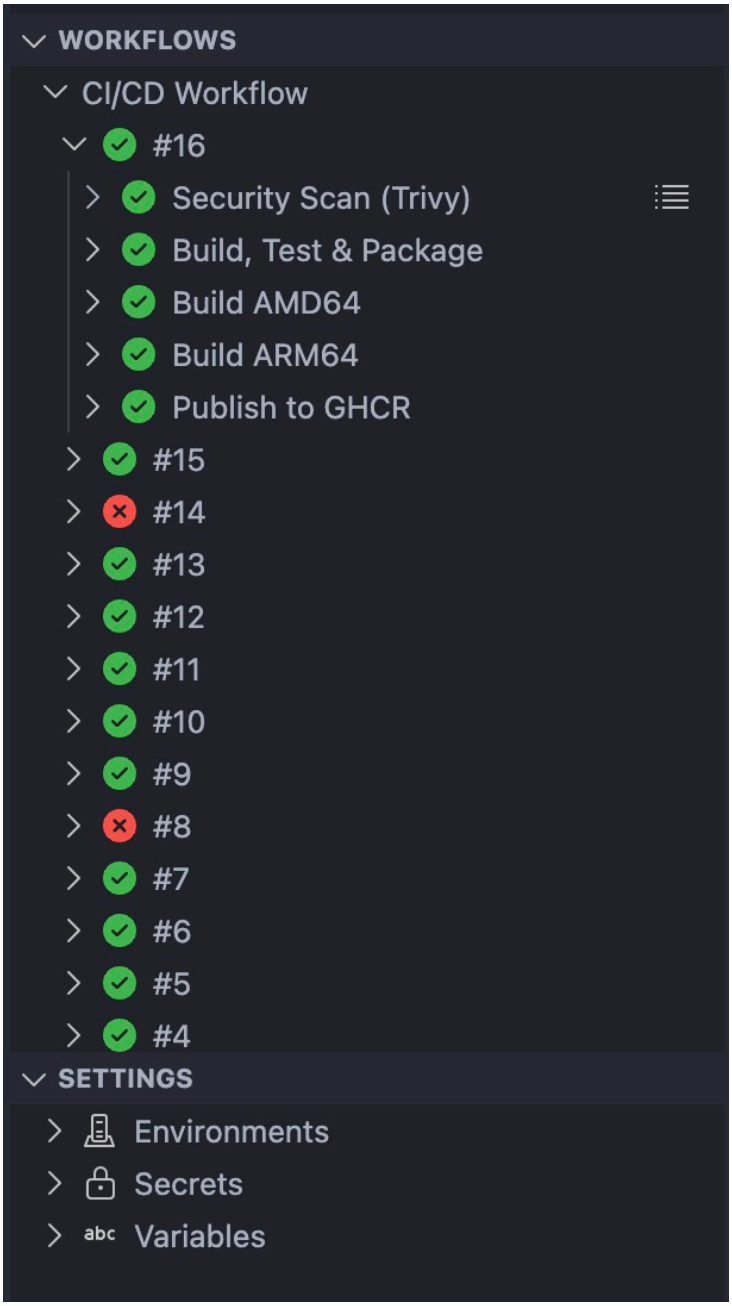

*Hình 3.16: Dịch vụ Web Proxy vproweb*

*3.5.2.7. Theo dõi quá trình triển khai tự động*

Sau khi image ứng dụng được phát hành, quá trình triển khai có thể được theo dõi qua kết quả workflow và log của Watchtower. Log cần thể hiện các bước phát hiện image mới, dừng container vproapp và khởi tạo lại container với cấu hình đã cập nhật.

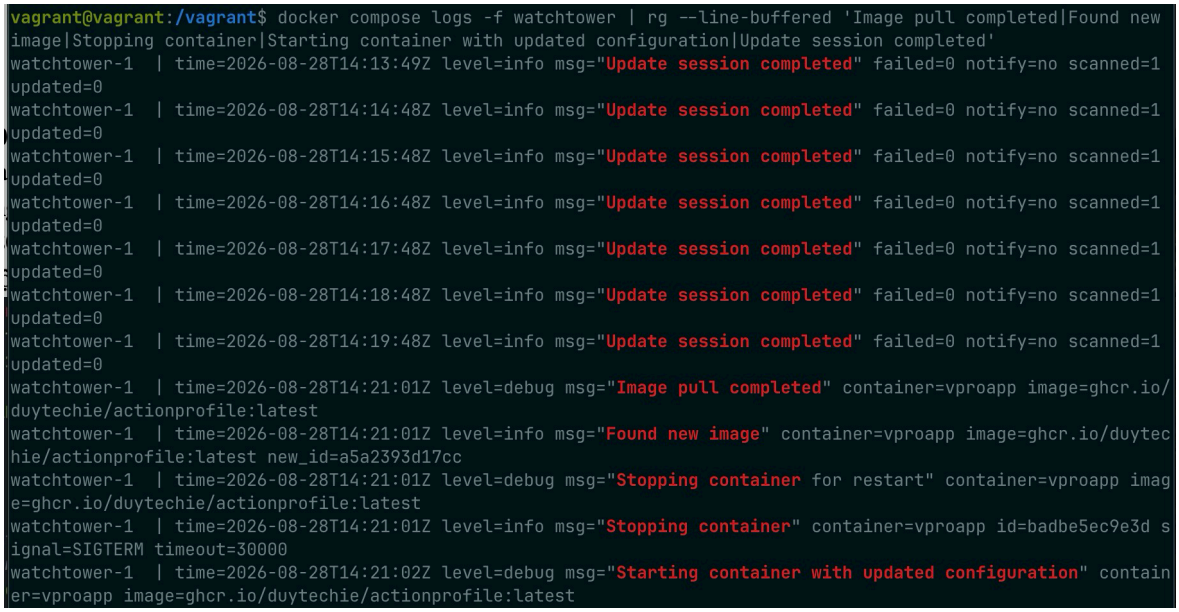



*Hình 3.17: Kết quả thực thi workflow triển khai ứng dụng*




*Hình 3.18: Kết quả Workflow trong VSCode*

Log của Watchtower được sử dụng để xác nhận hệ thống đã kéo image mới và cập nhật container vproapp thành công.




*Hình 3.19: Log của Watchtower khi cập nhật container vproapp*

# CHƯƠNG IV: ĐÁNH GIÁ KIỂM THỬ

## 4.1. Mục tiêu kiểm thử

Kiểm thử quy trình CI/CD đã xây dựng cho ứng dụng VProfile để xác nhận một thay đổi mã nguồn có được tích hợp, kiểm tra, đóng gói, phát hành và triển khai đúng theo luồng tự động hay không.

- **Phát hiện lỗi cấu hình:** Kiểm tra các lỗi có thể làm workflow không được kích hoạt, Maven không tạo được artifact, image không được phát hành hoặc container không được cập nhật.
- **Đối chiếu yêu cầu:** Đánh giá các kết quả thực tế với yêu cầu đặt ra cho pipeline: build một lần, quét bảo mật, build đa kiến trúc, công bố image lên GHCR và tự động cập nhật dịch vụ vproapp.
- **Quan sát log file:** Ghi nhận log Maven, kết quả GitHub Actions, artifact, manifest image và log Watchtower để phục vụ việc đánh giá, bảo trì và khắc phục sự cố về sau.

## 4.2. Phạm vi kiểm thử

Việc kiểm thử được thực hiện theo các nhóm sau:

- **Kiểm thử cấu hình và kích hoạt Workflow:** Xác nhận workflow được kích hoạt khi có push hoặc pull request vào nhánh main, đồng thời phân quyền đọc mã nguồn và ghi package được khai báo đúng.
- **Kiểm thử tích hợp liên tục:** Xác nhận Maven có thể biên dịch, chạy vòng đời test, đóng gói WAR và tải artifact vprofile-app lên GitHub Actions.
- **Kiểm thử chất lượng và bảo mật:** Xác nhận báo cáo Checkstyle và file trivy-results.json được sinh ra, có thể tải lại để đánh giá mã nguồn và thư viện phụ thuộc.
- **Kiểm thử phát hành và triển khai:** Xác nhận image AMD64 và ARM64 được gộp thành manifest trên GHCR; Watchtower phát hiện image latest mới và cập nhật đúng container vproapp.

Ngoài phạm vi kiểm thử:

- Kiểm thử khả năng chịu tải, kiểm thử hiệu năng chuyên sâu, kiểm thử xâm nhập và kiểm thử đầy đủ mọi chức năng giao diện người dùng chưa thuộc phạm vi của niên luận này.
- Workflow này chỉ thực hiện các bước test cơ bản. Chưa thực hiện các quy trình test và security test đầy đủ và chuyên nghiệp.

### 4.3. Kế hoạch kiểm thử

Các nội dung được kiểm thử:

- **Kích hoạt pipeline:** Sự kiện push và pull request vào nhánh main.
- **Build và artifact:** Lệnh mvn install, file WAR tại target và artifact vprofile-app.
- **Quét và báo cáo:** Job Security Scan, file trivy-results.json và báo cáo Checkstyle.
- **Image đa kiến trúc:** Job BUILD_AMD64, BUILD_ARM64 và PUBLISH_TO_GHCR cùng các tag commit, latest trên GHCR.
- **Triển khai tự động:** Chuỗi log Watchtower từ lúc phát hiện image mới đến khi khởi tạo lại vproapp.

### 4.4. Các bước kiểm thử và kết quả ghi nhận

Lúc này server Ubuntu để dựng trang web Java đã được khởi động sẵn.

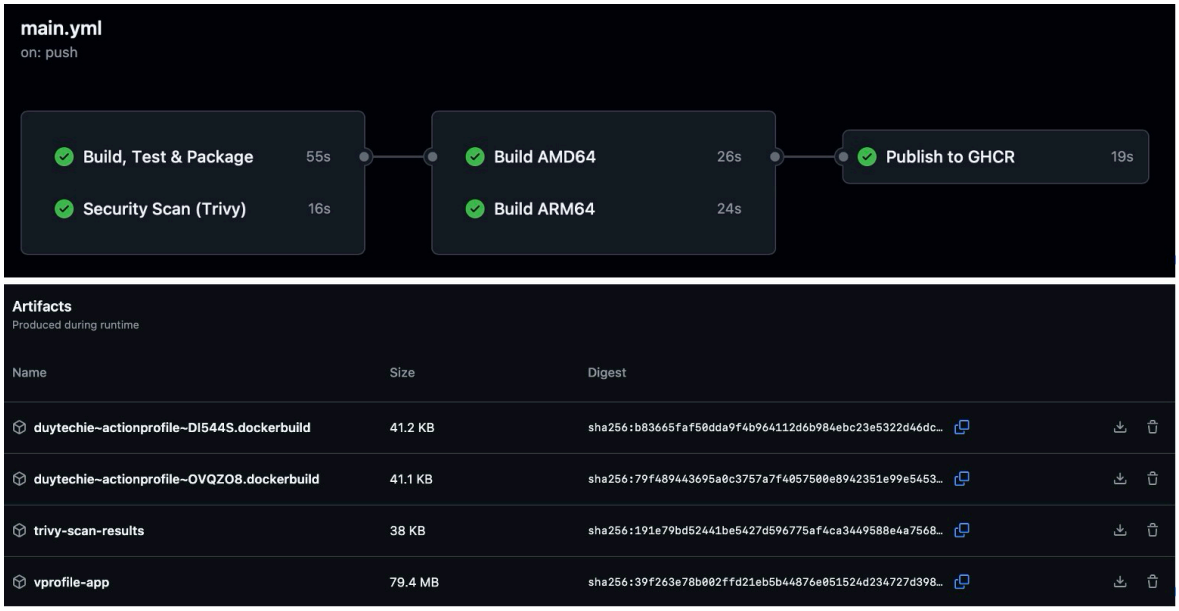
Mã nguồn được thay đổi và đẩy lên Github đồng thời kích hoạt Github Workflow.




*Hình 4.1: Git commit*

#### 4.4.1. Kiểm thử workflow và artifact

Github Workflow được kích hoạt. Kết quả mong đợi là các job hiển thị Success, **artifact vprofile-app** và **trivy-scan-results** có thể tải về, còn manifest đa kiến trúc được tạo sau khi hai image theo digest hoàn tất.




*Hình 4.2: Summary của GitHub Actions và danh sách Artifacts*

### 4.4.2. Kiểm thử build và test Maven

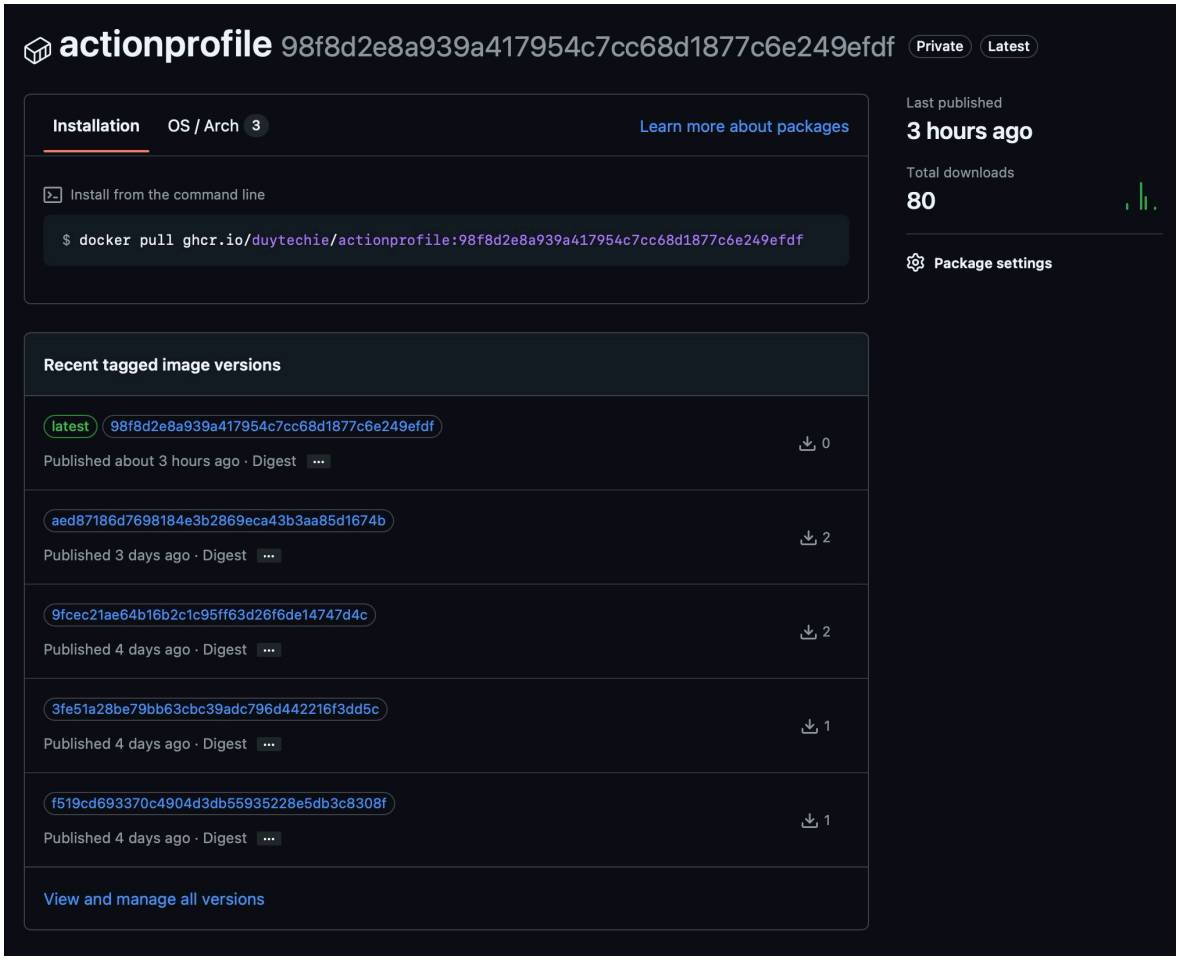
Kịch bản chạy lệnh mvn test nhằm xác nhận dự án có thể biên dịch và các test case được Maven nhận diện. Kết quả ghi nhận BUILD SUCCESS nhưng Tests run: 0, Failures: 0, Errors: 0.

```
11641 [INFO]  T E S T S
11642 [INFO] -------------------------------------------------------
11643 [INFO] Running com.visualpathit.account.controllerTest.SampleTest
11644 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in com.visualpathit.account.controllerTest.SampleTest
```

*Hình 4.3: Kết quả mvn test*

### 4.4.3. Kiểm thử phát hành image đa kiến trúc

Kịch bản kiểm tra package trên GHCR xác nhận cùng một tag latest có thể tham chiếu đến image linux/amd64 và linux/arm64.

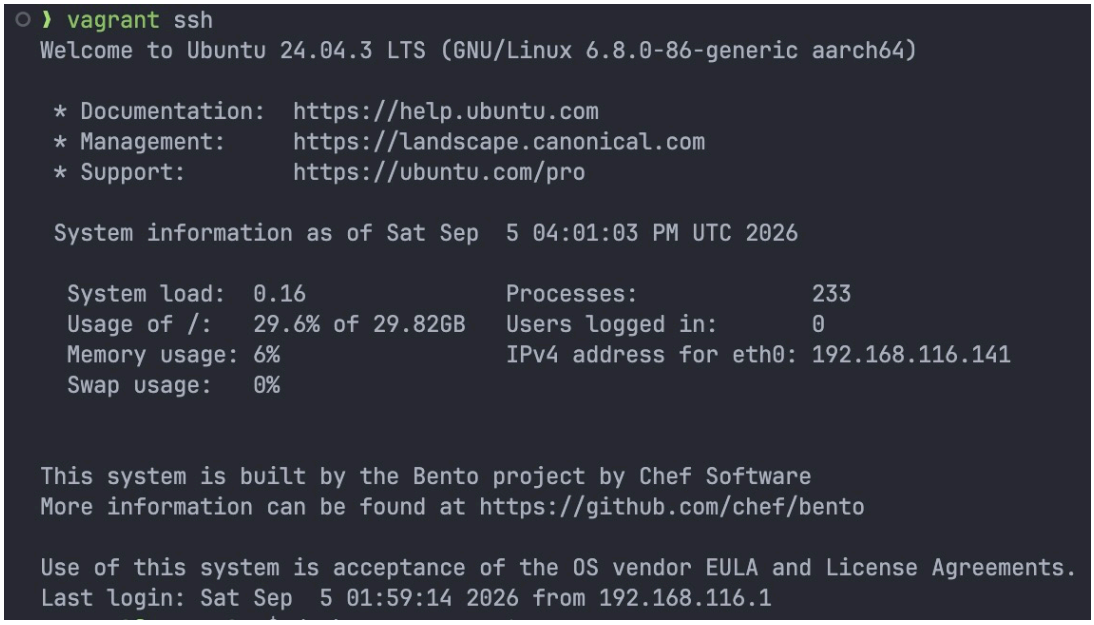



*Hình 4.4: Package trên GHCR*

### 4.4.4. Kiểm thử cập nhật tự động bằng Watchtower

Sau đó nếu workflow chạy thành công thì image latest mới đã xuất hiện trên GHCR. Kết quả mong đợi là Watchtower lần lượt kéo image, phát hiện image mới, dừng vproapp cũ và khởi động lại vproapp với cấu hình được giữ nguyên. Các dịch vụ hạ tầng khác không nằm trong phạm vi cập nhật của Watchtower.

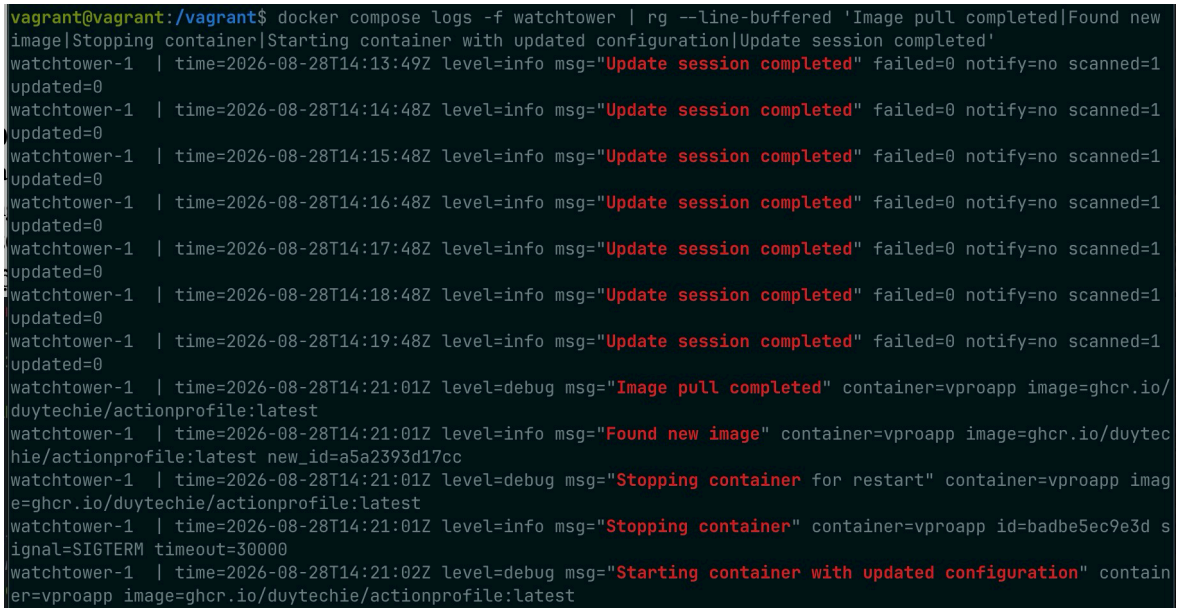
Thực hiện kết nối SSH vào server Ubuntu




*Hình 4.5: vagrant ssh*

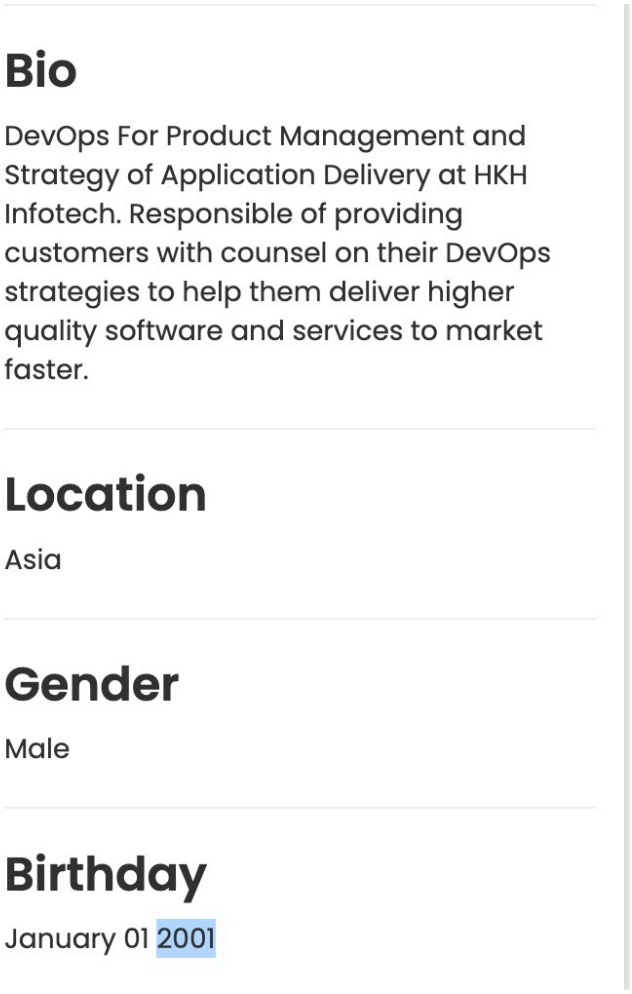
Kiểm tra log của watchtower bằng lệnh sau:

```
docker compose logs -f watchtower | rg --line-buffered 'Image pull completed|Found new image|Stopping container|Starting container with updated configuration|Update session completed'
```




*Hình 4.6: Log Watchtower*

Sau đó người dùng sẽ nhận được bản cập nhật mới một cách tự động sau lần refresh lại trang web




*Hình 4.7: Phiên bản web mới được cập nhật*

### 4.4.5. Đánh giá kiểm thử

Quy trình đã đáp ứng được mục tiêu tự động hóa việc build, test, security test, phát hành image đa kiến trúc và triển khai lại dịch vụ của ứng dụng web.

# TÀI LIỆU THAM KHẢO

vprofile-project
https://github.com/hkhcoder/vprofile-project

CICD là gì?
https://200lab.io/blog/ci-cd-la-gi

Push to GHCR
https://github.com/marketplace/actions/push-to-ghcr

Login Action
https://github.com/docker/login-action

Continuous Integration and Continuous Delivery/Deployment
https://www.blackduck.com/glossary/what-is-cicd.html